4.2 Preprocessing Data

Sebelum pemodelan dilakukan, data melalui tahap preprocessing. Tahap ini bertujuan untuk meningkatkan kualitas data agar dapat digunakan oleh algoritma machine learning. Preprocessing meliputi pembersihan data, penanganan missing value, encoding variabel kategorik, scaling variabel numerik, serta penghapusan variabel yang berpotensi menyebabkan data leakage.

Variabel target dan variabel turunan yang berkaitan langsung dengan pembentukan target tidak dimasukkan ke dalam variabel prediktor. Langkah ini penting karena model tidak boleh mempelajari informasi yang secara langsung membentuk label target. Jika variabel tersebut tetap digunakan, hasil model dapat terlihat tinggi tetapi tidak menggambarkan kemampuan prediksi yang sebenarnya.

Data kategorik diubah ke dalam bentuk numerik melalui proses encoding agar dapat dibaca oleh model. Variabel numerik juga diproses melalui scaling untuk membuat rentang nilai lebih konsisten. Meskipun Random Forest dan XGBoost merupakan model berbasis pohon yang tidak terlalu bergantung pada scaling, proses ini tetap digunakan untuk menjaga konsistensi preprocessing dan memudahkan pembacaan hasil.

kodingnya : `import pandas as pd`

```
import numpy as np


# ==============================================================================
# 1. BACA DATA
# ==============================================================================
file_asli = "/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes 2/01_Rifaskes_PKM.xlsx"
print("Membaca data RIFASKES PKM...")
df = pd.read_excel(file_asli)
```

```
# 📊 TRACKER 1: JUMLAH AWAL
total_pkm_awal = len(df)

# ================================================================================
# 2. SELEKSI KOLOM VIP (Whitelist)
# ================================================================================
kolom_dasar = ['ID_cases', 'B1R1', 'B1R2']

variabel_y = [
   'B1281R4K3', 'B1281R265K3', 'B1281R5K3', 'B1281R12K3', 'B1281R165K3', 'B1281R13K3', 'B1281R14K3', 'B1281R22K3',
   'B1281R11K3', 'B1281R41K3', 'B1281R51K3', 'B1281R58K3', 'B1281R63K3', 'B1281R65K3', 'B1281R2K3', 'B1281R361K3', 'B1281R115K3',
   'B1281R132K3', 'B1281R117K3', 'B1281R121K3', 'B1281R215K3', 'B1281R193K3', 'B1281R171K3', 'B1281R185K3', 'B1281R202K3',
   'B1281R211K3', 'B1281R217K3', 'B1281R233K3', 'B1281R243K3', 'B1281R250K3', 'B1281R256K3', 'B1281R257K3',
   'B1281R297K3', 'B1281R183K3', 'B1281R184K3', 'B1281R248K3', 'B1281R312K3', 'B1281R322K3', 'B1281R359K3', 'B1281R367K3'
]

variabel_x = [
   'B3R3', 'B3R10', 'B3R9B', 'B3R6', 'B3R11E', 'B3R13A', 'B14R2', 'B4R14', 'B4R16A', 'B14R23',
   'B14R15BTERSEDIA', 'B1282R6', 'B8R2K3N', 'B8R2K4N', 'B4R10B', 'B121R1', 'B121R3A', 'B121R3B', 'B121R4', 'B121R5', 'B121R7',
```

```
      'B121R10',  'B122R3',  'B122R4A',  'B122R4B',  'B122R5',  'B122R6',  'B13R2ATERSEDIA',  'B13R2BTERSEDIA',  'B13R2CTERSEDIA',
'B13R2DTERSEDIA', 'B13R2ETERSEDIA',

    'B1282R1', 'B1282R2', 'B1282R3', 'B1282R4', 'B1282R5', 'B6R1LK2', 'B6R1LK4', 'B122R11', 'B122R11A', 'B122R11B', 'B122R11C',
'B122R11E', 'B122R11F', 'B122R11G',

   'B122R1', 'B14R13B', 'B4R3', 'B51R1B', 'B3R22E', 'B4R6A1K2', 'B4R6A2K2', 'B4R6A3K2', 'B3R24',

   'B3R40', 'B51R2DK2', 'B9R13A', 'B125R2', 'B125R1', 'B14R16', 'B14R7A', 'B126R4A',

   'B9R7AK2', 'B9R7BK2', 'B9R7CK2', 'B9R7DK2', 'B9R7HK2', 'B122R7A', 'B122R7B', 'B122R7C', 'B122R7D'

]


kolom_skip_logic = ['B3R8', 'B3R5A', 'B51R1A', 'B14R1A', 'B3R23A']


# Gabungkan dan hapus duplikat

kolom_vip = list(set(kolom_dasar + variabel_y + variabel_x + kolom_skip_logic))


print("\nMenyapu bersih kolom yang tidak penting...")

kolom_vip_yg_ada = [col for col in kolom_vip if col in df.columns]

df = df[kolom_vip_yg_ada]

print(f"Beres! Data sisa {len(df.columns)} kolom penting.")


# ================================================================================

# 3. EKSEKUSI SKIP LOGIC & IMPUTASI

# ================================================================================
```

```python
def aman_loc(col_kondisi, val_kondisi, col_target, val_target):
    if col_kondisi in df.columns and col_target in df.columns:
        mask = (df[col_kondisi] == val_kondisi) & (df[col_target].isna())
        df.loc[mask, col_target] = val_target


print("\n--- Eksekusi Skip Logic ---")
aman_loc('B3R8', 3, 'B3R9B', 0)
aman_loc('B3R5A', 3, 'B3R6', 0)
aman_loc('B14R1A', 2, 'B14R2', 0)
aman_loc('B3R3', 2, 'B4R16A', 0)
aman_loc('B121R1', 2, 'B121R3A', 0)
aman_loc('B121R1', 2, 'B121R3B', 0)
aman_loc('B121R1', 2, 'B121R4', 0)
aman_loc('B121R1', 2, 'B121R5', 0)
aman_loc('B121R7', 1, 'B121R10', 2)
aman_loc('B122R3', 4, 'B122R4A', 0)
aman_loc('B122R3', 4, 'B122R4B', 0)
aman_loc('B122R3', 4, 'B122R6', 0)
aman_loc('B1282R1', 2, 'B1282R2', 0)
aman_loc('B1282R3', 2, 'B1282R4', 0)
aman_loc('B6R1LK2', 3, 'B6R1LK4', 0)
```

```
aman_loc('B122R11', 2, 'B122R11A', 0)

aman_loc('B122R11', 2, 'B122R11B', 0)

aman_loc('B122R11', 2, 'B122R11C', 0)

aman_loc('B122R11', 2, 'B122R11E', 0)

aman_loc('B122R11', 2, 'B122R11F', 0)

aman_loc('B122R11', 2, 'B122R11G', 0)

aman_loc('B14R1A', 2, 'B14R13B', 0)

aman_loc('B14R1A', 2, 'B14R16', 0)

aman_loc('B14R1A', 2, 'B14R7A', 0)

aman_loc('B122R3', 4, 'B122R7A', 0)

aman_loc('B122R3', 4, 'B122R7B', 0)

aman_loc('B122R3', 4, 'B122R7C', 0)

aman_loc('B122R3', 4, 'B122R7D', 0)

aman_loc('B51R1A', 3, 'B51R1B', 0)

aman_loc('B3R23A', 2, 'B3R24', 0)


print(">> Mengamankan E-Purchasing (diisi dengan 0)")

if 'B122R5' in df.columns: df['B122R5'] = df['B122R5'].fillna(0)

if 'B122R6' in df.columns: df['B122R6'] = df['B122R6'].fillna(0)


nan_sekarang = df.isna().sum()
```

```
kolom_nan_minor = nan_sekarang[nan_sekarang > 0].index.tolist()

pkm_sebelum_drop_nan = len(df)

df = df.dropna(subset=kolom_nan_minor)

pkm_gugur_nan_minor = pkm_sebelum_drop_nan - len(df)


# ================================================================================

# 4. HAPUS BARIS PERMANEN (Buat Siluman Fatal)

# ================================================================================

print("\nMenghapus baris siluman fatal...")

aturan_hapus_baris = {

   'B4R14': [888888, 777777, 999997],

   'B4R16A': [777777, 888888, 999997],

   'B14R15BTERSEDIA': [0, 8, 9, 5],

   'B8R2K3N': [88],

   'B121R4': [8],

   'B1282R2': [888, 88, 77],

   'B1282R4': [-4, 88, 77, 888, 96],

   'B9R13A': [88, 99, 97],

   'B14R7A': [3, 7, 8, 9],

   'B9R7DK2': [88]

}
```

```python
pkm_sebelum_fatal = len(df)
for kolom, angka_fatal in aturan_hapus_baris.items():
    if kolom in df.columns:
        df = df[~df[kolom].isin(angka_fatal)]

pkm_gugur_fatal = pkm_sebelum_fatal - len(df)

print("Mengubah siluman B14R23 jadi -1...")
if 'B14R23' in df.columns:
    df['B14R23'] = df['B14R23'].replace([88888, 77777, 99999, 99997, ], -1)

# ==============================================================================
# 5. FEATURE ENGINEERING: DAPUR RACIK VARIABEL GABUNGAN
# ==============================================================================
print("\n👨‍🍳 Memulai proses peracikan variabel baru...")

# --- B. PENGIRIMAN RKO ---
if 'B121R3A' in df.columns and 'B121R3B' in df.columns:
    kondisi_rko = [
        (df['B121R3A'] == 1) & (df['B121R3B'] == 1),
```

```
        (df['B121R3A'] == 1) & (df['B121R3B'].isin([2, 0])),
        (df['B121R3A'].isin([2, 0])) & (df['B121R3B'] == 1),
        (df['B121R3A'].isin([2, 0])) & (df['B121R3B'].isin([2, 0]))
    ]
    df['X_Pengiriman_RKO'] = np.select(kondisi_rko, [3, 1, 2, 0], default=np.nan)

# --- E. METODE PEMBELIAN OBAT KAPITASI ---
if 'B122R4A' in df.columns and 'B122R4B' in df.columns:
    kondisi_metode = [
        (df['B122R4A'] == 1) & (df['B122R4B'] == 1),
        ((df['B122R4A'] == 1) & df['B122R4B'].isin([2, 0])) | (df['B122R4A'].isin([2, 0]) & (df['B122R4B'] == 1)),
        df['B122R4A'].isin([2, 0]) & df['B122R4B'].isin([2, 0]),
        df['B122R4A'].isin([8]) | df['B122R4B'].isin([8])
    ]
    df['X_Metode_Pembelian_Kapitasi'] = np.select(kondisi_metode, [2, 1, 0, -1], default=np.nan)

# --- PEMENUHAN E-PURCHASING ---
kondisi_pemenuhan = [
    (df['B122R5'] == 1),
    (df['B122R5'] == 2) & (df['B122R6'] == 4),
    (df['B122R5'] == 2) & (df['B122R6'] == 3),
```

```
    (df['B122R5'] == 2) & (df['B122R6'] == 2),
    (df['B122R5'] == 2) & (df['B122R6'] == 1),
    (df['B122R5'] == 0)
]
df['X_Persentase_Pemenuhan_EPurchasing'] = np.select(kondisi_pemenuhan, [5, 4, 3, 2, 1, 0], default=0)
if 'B122R5' in df.columns: df = df.drop(columns=['B122R5', 'B122R6'])


# --- C. DANA UMUM ---
kolom_dana_umum = ['B13R2ATERSEDIA', 'B13R2BTERSEDIA', 'B13R2CTERSEDIA']
if set(kolom_dana_umum).issubset(df.columns):
        df['Total_Dana_Umum'] = (df['B13R2ATERSEDIA'] == 1).astype(int) + (df['B13R2BTERSEDIA'] == 1).astype(int) +
(df['B13R2CTERSEDIA'] == 1).astype(int)
       kondisi_dana_umum = [(df['Total_Dana_Umum'] == 3), (df['Total_Dana_Umum'] == 2), (df['Total_Dana_Umum'] == 1),
(df['Total_Dana_Umum'] == 0)]
    df['X_Sumber_Dana_Umum'] = np.select(kondisi_dana_umum, [3, 2, 1, 0], default=np.nan)
    df = df.drop(columns=['Total_Dana_Umum'])


# --- F. RUANGAN FARMASI ---
if 'B6R1LK2' in df.columns and 'B6R1LK4' in df.columns:
    kondisi_ruangan = [
        (df['B6R1LK2'] == 1) & (df['B6R1LK4'] == 1), (df['B6R1LK2'] == 1) & (df['B6R1LK4'] == 2),
        (df['B6R1LK2'] == 1) & (df['B6R1LK4'] == 3), (df['B6R1LK2'] == 2) & (df['B6R1LK4'] == 1),
```

```
        (df['B6R1LK2'] == 2) & (df['B6R1LK4'] == 2), (df['B6R1LK2'] == 2) & (df['B6R1LK4'] == 3),
        (df['B6R1LK2'] == 3)
    ]
    df['X_Kondisi_Ruang_Farmasi'] = np.select(kondisi_ruangan, [6, 5, 4, 3, 2, 1, 0], default=np.nan)


# --- G. KUALITAS GUDANG OBAT ---
kolom_gudang = ['B122R11A', 'B122R11B', 'B122R11C', 'B122R11E', 'B122R11F', 'B122R11G']
if set(kolom_gudang).issubset(df.columns):
    is_nan = df['B122R11A'].isna()
    df['Skor_Gudang'] = (
        (df['B122R11A'] == 1).astype(int) + (df['B122R11B'] == 1).astype(int) +
        (df['B122R11C'] == 1).astype(int) + (df['B122R11E'] == 1).astype(int) +
        (df['B122R11F'] == 1).astype(int) + (df['B122R11G'] == 1).astype(int)
    )
    kondisi_gudang = [
        (df['Skor_Gudang'] == 6) & ~is_nan, (df['Skor_Gudang'].isin([4, 5])) & ~is_nan,
        (df['Skor_Gudang'].isin([2, 3])) & ~is_nan, (df['Skor_Gudang'].isin([0, 1])) & ~is_nan, is_nan
    ]
    df['X_Kualitas_Gudang'] = np.select(kondisi_gudang, [4, 3, 2, 1, 0], default=np.nan)
    df = df.drop(columns=kolom_gudang + ['Skor_Gudang'])
```

```python
# --- A. DANA BPJS ---
if 'B13R2DTERSEDIA' in df.columns and 'B13R2ETERSEDIA' in df.columns:
    kondisi_bpjs = [
        (df['B13R2DTERSEDIA'] == 1) & (df['B13R2ETERSEDIA'] == 1),
        (df['B13R2DTERSEDIA'] == 1) & (df['B13R2ETERSEDIA'].isin([2, 0])),
        (df['B13R2DTERSEDIA'].isin([2, 0])) & (df['B13R2ETERSEDIA'] == 1),
        (df['B13R2DTERSEDIA'].isin([2, 0])) & (df['B13R2ETERSEDIA'].isin([2, 0]))
    ]
    df['X_Sumber_Dana_BPJS'] = np.select(kondisi_bpjs, [3, 2, 1, 0], default=np.nan)

# --- H. PROPORSI KAPITASI ---
if 'B14R13B' in df.columns:
    df['B14R13B'] = df['B14R13B'].fillna(0)
    df['X_Proporsi_Kapitasi'] = np.where(df['B14R13B'] <= 40.0, 1, 0)
    df = df.drop(columns=['B14R13B'])

# --- I. SISTEM PENCATATAN ---
kolom_sistem = ['B4R6A1K2', 'B4R6A2K2', 'B4R6A3K2']
if set(kolom_sistem).issubset(df.columns):
    cond_tidak_ada = df[kolom_sistem].isin([4, 0, np.nan]).all(axis=1)
    ada_campuran = (df[kolom_sistem] == 2).any(axis=1)
```

```
    ada_elektronik = (df[kolom_sistem] == 1).any(axis=1)
    ada_manual = (df[kolom_sistem] == 3).any(axis=1)
    cond_campuran_final = ada_campuran | (ada_elektronik & ada_manual)
    cond_elektronik_final = ada_elektronik & ~cond_campuran_final & ~ada_manual
    cond_manual_final = ada_manual & ~cond_campuran_final & ~ada_elektronik
    kondisi_sistem = [cond_elektronik_final, cond_manual_final, cond_campuran_final, cond_tidak_ada]
    df['X_Sistem_Pencatatan'] = np.select(kondisi_sistem, [3, 2, 1, 0], default=np.nan)

# --- K. KERUSAKAN OBAT ---
if 'B125R1' in df.columns and 'B125R2' in df.columns:
    kondisi_rusak = [
        (df['B125R1'] == 1) & (df['B125R2'] == 1),
        (df['B125R1'] == 1) & (df['B125R2'].isin([2, 0])),
        (df['B125R1'].isin([2, 0])) & (df['B125R2'] == 1),
        (df['B125R1'].isin([2, 0])) & (df['B125R2'].isin([2, 0]))
    ]
    df['X_Kerusakan_Obat'] = np.select(kondisi_rusak, [0, 1, 2, 3], default=np.nan)

# --- L. KEPEMILIKAN KENDARAAN ---
kolom_kendaraan = ['B9R7AK2', 'B9R7BK2', 'B9R7CK2', 'B9R7DK2', 'B9R7HK2']
if set(kolom_kendaraan).issubset(df.columns):
```

```
   punya_roda_4 = (df['B9R7AK2'].between(1, 87)) | (df['B9R7BK2'].between(1, 87)) | (df['B9R7CK2'].between(1, 87))

   punya_roda_2 = (df['B9R7DK2'].between(1, 87))

      kondisi_kendaraan  =  [(punya_roda_4  &  punya_roda_2),  (punya_roda_4  &  ~punya_roda_2),  (~punya_roda_4  &  punya_roda_2),
(~punya_roda_4 & ~punya_roda_2)]

   df['X_Kepemilikan_Kendaraan'] = np.select(kondisi_kendaraan, [3, 2, 1, 0], default=np.nan)


# --- D. INTENSITAS KAPITASI ---

kolom_kapitasi = ['B122R7A', 'B122R7B', 'B122R7C', 'B122R7D']

if set(kolom_kapitasi).issubset(df.columns):

     df['X_Intensitas_Kapitasi']  =  (df['B122R7A']  ==  1).astype(int)  +  (df['B122R7B']  ==  1).astype(int)  +  (df['B122R7C']  ==
1).astype(int) + (df['B122R7D'] == 1).astype(int)

   df = df.drop(columns=kolom_kapitasi)


# ================================================================================

# FINALISASI: BUANG SEMUA KOLOM BAHAN BAKU

# ================================================================================

kolom_dibuang = [

   'B13R2DTERSEDIA', 'B13R2ETERSEDIA', 'B121R3A', 'B121R3B', 'B13R2ATERSEDIA', 'B13R2BTERSEDIA', 'B13R2CTERSEDIA',

   'B122R4A', 'B122R4B', 'B6R1LK2', 'B6R1LK4', 'B122R11A', 'B122R11B',

   'B122R11C', 'B122R11E', 'B122R11F', 'B122R11G', 'B4R6A1K2', 'B4R6A2K2', 'B4R6A3K2',

   'B125R1', 'B125R2', 'B9R7AK2', 'B9R7BK2', 'B9R7CK2', 'B9R7DK2', 'B9R7HK2'

]
```

```python
df = df.drop(columns=[c for c in kolom_dibuang if c in df.columns])


# ==============================================================================
# 6. BUAT VARIABEL TARGET Y (PUSKESMAS)
# ==============================================================================
if 'B1281R4K3' in df.columns and 'B1281R265K3' in df.columns: df['Y_Obat_Cacing'] = df[['B1281R4K3', 'B1281R265K3']].min(axis=1)
if 'B1281R12K3' in df.columns and 'B1281R165K3' in df.columns: df['Y_Hipertensi'] = df[['B1281R12K3', 'B1281R165K3']].min(axis=1)
if 'B1281R115K3' in df.columns and 'B1281R132K3' in df.columns: df['Y_Diuretik'] = df[['B1281R115K3', 'B1281R132K3']].min(axis=1)
if 'B1281R121K3' in df.columns and 'B1281R215K3' in df.columns: df['Y_Diabetes'] = df[['B1281R121K3', 'B1281R215K3']].min(axis=1)
kolom_mata = [c for c in ['B1281R183K3', 'B1281R184K3', 'B1281R248K3'] if c in df.columns]
if kolom_mata: df['Y_Mata'] = df[kolom_mata].min(axis=1)


mapping_y_tunggal = {
   'B1281R5K3': 'Y_Alopurinol', 'B1281R13K3': 'Y_Amoksisilin_Sirup', 'B1281R14K3': 'Y_Amoksisilin_Tablet',
   'B1281R22K3': 'Y_Antasida', 'B1281R11K3': 'Y_Amitriptilin', 'B1281R41K3': 'Y_Asiklovir',
   'B1281R51K3': 'Y_Betametason_Krim', 'B1281R58K3': 'Y_Deksametason', 'B1281R63K3': 'Y_Diazepam_Injeksi',
   'B1281R65K3': 'Y_Diazepam_Tablet', 'B1281R2K3': 'Y_Adrenalin_Injeksi', 'B1281R361K3': 'Y_VitK_Injeksi',
   'B1281R117K3': 'Y_Oralit', 'B1281R193K3': 'Y_Kotrimoksazol', 'B1281R171K3': 'Y_Ketokonazol',
   'B1281R185K3': 'Y_CTM', 'B1281R202K3': 'Y_Lidokain_Injeksi', 'B1281R211K3': 'Y_MgSO4_Injeksi',
   'B1281R217K3': 'Y_Metilergometrin_Injeksi', 'B1281R233K3': 'Y_Natrium_Diklofenak',
   'B1281R243K3': 'Y_OAT_Dewasa', 'B1281R250K3': 'Y_Oksitosin_Injeksi', 'B1281R256K3': 'Y_Parasetamol_Sirup',
```

```
    'B1281R257K3': 'Y_Parasetamol_Tablet', 'B1281R297K3': 'Y_Salbutamol', 'B1281R312K3': 'Y_Simvastatin',
    'B1281R322K3': 'Y_TTD_Besi', 'B1281R359K3': 'Y_VitB6', 'B1281R367K3': 'Y_Zink'
}
for col_asli, col_baru in mapping_y_tunggal.items():
    if col_asli in df.columns: df[col_baru] = df[col_asli]

target_y_potensial = ['Y_Obat_Cacing', 'Y_Hipertensi', 'Y_Diuretik', 'Y_Diabetes', 'Y_Mata'] + list(mapping_y_tunggal.values())
kolom_y_yg_sukses_dibuat = [col for col in target_y_potensial if col in df.columns]

pkm_sebelum_y = len(df)
df = df.dropna(subset=kolom_y_yg_sukses_dibuat)
pkm_gugur_y = pkm_sebelum_y - len(df)

for col in kolom_y_yg_sukses_dibuat: df[col] = df[col].replace(2, 0)
df['Total_Kelompok_Obat_PKM'] = df[kolom_y_yg_sukses_dibuat].sum(axis=1)
df['Y_Final_PKM'] = np.where(df['Total_Kelompok_Obat_PKM'] >= 32, 1, 0)

# ================================================================================
# 7. BACA DATA DINKES & MERGE SEBAGAI VARIABEL X
# ================================================================================
print("\nMemproses Data Dinkes dan Merging...")
```

```
file_dinkes = "/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes 2/01_Rifaskes_Dinkes.xlsx"

df_dinkes = pd.read_excel(file_dinkes)

df_dinkes.columns = df_dinkes.columns.str.strip()


if 'B66R4K3' in df_dinkes.columns and 'B66R265K3' in df_dinkes.columns: df_dinkes['X_Obat_Cacing'] = df_dinkes[['B66R4K3',
'B66R265K3']].min(axis=1)

if 'B66R12K3' in df_dinkes.columns and 'B66R165K3' in df_dinkes.columns: df_dinkes['X_Hipertensi'] = df_dinkes[['B66R12K3',
'B66R165K3']].min(axis=1)

if 'B66R115K3' in df_dinkes.columns and 'B66R132K3' in df_dinkes.columns: df_dinkes['X_Diuretik'] = df_dinkes[['B66R115K3',
'B66R132K3']].min(axis=1)

if 'B66R121K3' in df_dinkes.columns and 'B66R215K3' in df_dinkes.columns: df_dinkes['X_Diabetes'] = df_dinkes[['B66R121K3',
'B66R215K3']].min(axis=1)

kolom_mata_d = [c for c in ['B66R183K3', 'B66R184K3', 'B66R248K3'] if c in df_dinkes.columns]

if kolom_mata_d: df_dinkes['X_Mata'] = df_dinkes[kolom_mata_d].min(axis=1)


mapping_x_tunggal = {

   'B66R5K3': 'X_Alopurinol', 'B66R13K3': 'X_Amoksisilin_Sirup', 'B66R14K3': 'X_Amoksisilin_Tablet',

   'B66R22K3': 'X_Antasida', 'B66R11K3': 'X_Amitriptilin', 'B66R41K3': 'X_Asiklovir',

   'B66R51K3': 'X_Betametason_Krim', 'B66R58K3': 'X_Deksametason', 'B66R63K3': 'X_Diazepam_Injeksi',

   'B66R65K3': 'X_Diazepam_Tablet', 'B66R2K3': 'X_Adrenalin_Injeksi', 'B66R361K3': 'X_VitK_Injeksi',

   'B66R117K3': 'X_Oralit', 'B66R193K3': 'X_Kotrimoksazol', 'B66R171K3': 'X_Ketokonazol',

   'B66R185K3': 'X_CTM', 'B66R202K3': 'X_Lidokain_Injeksi', 'B66R211K3': 'X_MgSO4_Injeksi',
```

```
    'B66R217K3': 'X_Metilergometrin_Injeksi', 'B66R233K3': 'X_Natrium_Diklofenak',
    'B66R243K3': 'X_OAT_Dewasa', 'B66R250K3': 'X_Oksitosin_Injeksi', 'B66R256K3': 'X_Parasetamol_Sirup',
    'B66R257K3': 'X_Parasetamol_Tablet', 'B66R297K3': 'X_Salbutamol', 'B66R312K3': 'X_Simvastatin',
    'B66R322K3': 'X_TTD_Besi', 'B66R359K3': 'X_VitB6', 'B66R367K3': 'X_Zink'
}
for col_asli, col_baru in mapping_x_tunggal.items():
    if col_asli in df_dinkes.columns: df_dinkes[col_baru] = df_dinkes[col_asli]

target_x_potensial = ['X_Obat_Cacing', 'X_Hipertensi', 'X_Diuretik', 'X_Diabetes', 'X_Mata'] + list(mapping_x_tunggal.values())
kolom_x_sukses = [col for col in target_x_potensial if col in df_dinkes.columns]

for col in kolom_x_sukses: df_dinkes[col] = df_dinkes[col].replace(2, 0)
df_dinkes['Total_Kelompok_Obat_Dinkes'] = df_dinkes[kolom_x_sukses].sum(axis=1)
df_dinkes['X_Ketersediaan_Dinkes'] = np.where(df_dinkes['Total_Kelompok_Obat_Dinkes'] >= 34, 1, 0)

# Merge berdasarkan provinsi & kabupaten
df['B1R1_str'] = df['B1R1'].astype(str)
df['B1R2_str'] = df['B1R2'].astype(str)
df['KODE_KAB_PUSKESMAS'] = df['B1R1_str'] + df['B1R2_str'].str.zfill(2)

df_dinkes['propkab'] = df_dinkes['propkab'].astype(str)
```

```python
df_dinkes_subset = df_dinkes[['propkab', 'X_Ketersediaan_Dinkes']]

df = pd.merge(df, df_dinkes_subset, left_on='KODE_KAB_PUSKESMAS', right_on='propkab', how='left')

df = df.drop(columns=['B1R1_str', 'B1R2_str', 'KODE_KAB_PUSKESMAS', 'propkab'])


# ================================================================================

# 8. SAPU BERSIH KOLOM BAHAN BAKU & BOCORAN ANTI-NYONTEK

# ================================================================================

df_bersih = df.copy()


# Buang bahan mentah obat PKM (variabel_y asli)

kolom_obat_mentah_pkm = variabel_y

df_bersih = df_bersih.drop(columns=[c for c in kolom_obat_mentah_pkm if c in df_bersih.columns])


# Buang Y perantara (Cuma disisain Y_Final_PKM)

kolom_bocoran = [col for col in df_bersih.columns if col.startswith('Y_') and col != 'Y_Final_PKM']

df_bersih = df_bersih.drop(columns=[c for c in kolom_bocoran if c in df_bersih.columns])


# Buang kolom syarat skip logic yang tadi lolos

sampah_sisa = ['B51R1A', 'B3R8', 'B3R5A', 'B14R1A', 'B3R23A']

df_bersih = df_bersih.drop(columns=[c for c in sampah_sisa if c in df_bersih.columns])
```

```python
# ==============================================================================
# 9. MAPPING NILAI ORDINAL (Mengubah ke Skala Linear)
# ==============================================================================
print("\nMelakukan mapping variabel...")
# Menggunakan .replace() agar nilai di luar kamus tidak berubah jadi NaN
if 'B3R6' in df_bersih.columns: df_bersih['B3R6'] = df_bersih['B3R6'].replace({3: 1, 1: 3})
if 'B3R3' in df_bersih.columns: df_bersih['B3R3'] = df_bersih['B3R3'].replace({2: 0})
if 'B3R10' in df_bersih.columns: df_bersih['B3R10'] = df_bersih['B3R10'].replace({3: 0, 2: 1, 1: 2})
if 'B14R2' in df_bersih.columns: df_bersih['B14R2'] = df_bersih['B14R2'].replace({2: 1, 1: 2})
if 'B1282R6' in df_bersih.columns: df_bersih['B1282R6'] = df_bersih['B1282R6'].replace({2: 0})
if 'B8R2K4N' in df_bersih.columns: df_bersih['B8R2K4N'] = df_bersih['B8R2K4N'].replace({3: 0, 2: 1, 1: 2})
if 'B4R10B' in df_bersih.columns: df_bersih['B4R10B'] = df_bersih['B4R10B'].replace({3: 0, 2: 1, 1: 2})
if 'B121R1' in df_bersih.columns: df_bersih['B121R1'] = df_bersih['B121R1'].replace({2: 0})
if 'B121R5' in df_bersih.columns: df_bersih['B121R5'] = df_bersih['B121R5'].replace({1: 2, 2: 1})
if 'B121R7' in df_bersih.columns: df_bersih['B121R7'] = df_bersih['B121R7'].replace({2: 0})
if 'B121R10' in df_bersih.columns: df_bersih['B121R10'] = df_bersih['B121R10'].replace({2: 0})
if 'B122R3' in df_bersih.columns: df_bersih['B122R3'] = df_bersih['B122R3'].replace({4: 0})
if 'B1282R1' in df_bersih.columns: df_bersih['B1282R1'] = df_bersih['B1282R1'].replace({2: 0})
if 'B1282R3' in df_bersih.columns: df_bersih['B1282R3'] = df_bersih['B1282R3'].replace({2: 0})
if 'B122R11' in df_bersih.columns: df_bersih['B122R11'] = df_bersih['B122R11'].replace({2: 0})
if 'B122R1' in df_bersih.columns: df_bersih['B122R1'] = df_bersih['B122R1'].replace({3: 0})
```

```python
if 'B4R3' in df_bersih.columns: df_bersih['B4R3'] = df_bersih['B4R3'].replace({3: 0, 2: 1, 1: 2})

if 'B51R1B' in df_bersih.columns: df_bersih['B51R1B'] = df_bersih['B51R1B'].replace({6: 1, 5: 2, 4: 3, 3: 4, 2: 5})

if 'B51R2DK2' in df_bersih.columns: df_bersih['B51R2DK2'] = df_bersih['B51R2DK2'].replace({2: 0})

if 'B3R22E' in df_bersih.columns: df_bersih['B3R22E'] = df_bersih['B3R22E'].replace({2: 0})

if 'B3R24' in df_bersih.columns: df_bersih['B3R24'] = df_bersih['B3R24'].replace({4: 1, 3: 2, 2: 3, 1: 4})

if 'B3R40' in df_bersih.columns: df_bersih['B3R40'] = df_bersih['B3R40'].replace({2: 0})

if 'B1282R5' in df_bersih.columns: df_bersih['B1282R5'] = df_bersih['B1282R5'].replace({2: 0})

if 'B14R16' in df_bersih.columns: df_bersih['B14R16'] = df_bersih['B14R16'].replace({2: 1, 1: 2})

if 'B126R4A' in df_bersih.columns: df_bersih['B126R4A'] = df_bersih['B126R4A'].replace({2: 0})

if 'B14R15BTERSEDIA' in df_bersih.columns: df_bersih['B14R15BTERSEDIA'] = df_bersih['B14R15BTERSEDIA'].replace({2: 0})


print("\n==================================================================")

print("🏆 LAPORAN TRACKER ELIMINASI PUSKESMAS 🏆")

print("==================================================================")

print(f"1. Total Puskesmas Awal          : {total_pkm_awal}")

print(f"2. Gugur karena NaN Minor (<5%)  : - {pkm_gugur_nan_minor}")

print(f"3. Gugur karena Siluman Fatal    : - {pkm_gugur_fatal}")

print(f"4. Gugur karena Target Y Kosong  : - {pkm_gugur_y}")

print("------------------------------------------------------------------")

print(f"🎯 SISA PUSKESMAS BERSIH         : {len(df_bersih)} Puskesmas")

print("==================================================================")
```

```
# ==============================================================================
# 10. SIMPAN EXCEL BERSIH!
# ==============================================================================
file_simpan = "/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes 2/01_Siap_MICE.xlsx"
df_bersih.to_excel(file_simpan, index=False)
print(f"\n📁 File berhasil disimpan di: {file_simpan}")
```

tabel desrkitpid numerik

=== TABEL 1: STATISTIK DESKRIPTIF DATA AWAL (SEBELUM) ===

	B3R11E	B3R13A	B4R14	B4R16A	B8R2K3N	B1282R2	B1282R4	B9R13A	B14R23
count	9761	9825	9831	4093	9828	2323	9503	9831	9830
mean	26943.1	2.64	54564.2	31837.2	0.54	6.25	8.24	9.82	23785.7
std	31156	2.61	167759	161836	2.04	21.32	10.67	8.83	29824.2
min	0	0	0	0	0	0	-4	0	0
25%	10442	1	5622	112	0	1	4	4	4563.25
50%	21229	2	14839	341	0	3	6	8	11256
75%	37188	4	29795.5	743	1	6	12	14	24657.5

| max | 1.41732e+06 | 60 | 999997 | 999997 | 88 | 888 | 888 | 99 | 99999 |

==========

=== TABEL 2: STATISTIK DESKRIPTIF DATA BERSIH (SESUDAH) ===

	B3R11E	B3R13A	B4R14	B4R16A	B8R2K3N	B1282R2	B1282R4	B9R13A	B14R23
count	9061	9061	9061	9061	9061	9061	9061	9061	9061
mean	27147.2	2.67	22757.4	446.25	0.51	1.08	7.85	9.96	10526.5
std	28842.5	2.53	41925	7284.47	1.02	2.9	4.6	8.28	12097.9
min	0	0	0	0	0	0	0	0	-1
25%	10810	1	5395	0	0	0	4	4	1375
50%	21753	2	13878	0	0	0	6	8	7393
75%	37476	4	27521	234	1	0	12	14	15287
max	1.41732e+06	60	999430	402045	55	42	66	90	99431

4.3 Pembagian Data dan Evaluasi Model

Data dibagi menjadi data latih dan data uji. Data latih digunakan untuk melatih model dalam mempelajari pola hubungan antar variabel, sedangkan data uji digunakan untuk mengukur kemampuan model pada data yang belum pernah dilihat sebelumnya. Pembagian data dilakukan dengan pendekatan stratified random sampling berdasarkan variabel target atau stratify=y.

Stratifikasi berdasarkan target dipilih karena penelitian ini merupakan kasus klasifikasi biner. Teknik ini menjaga proporsi kelas 0 dan 1 agar tetap seimbang pada data latih dan data uji. Dengan cara ini, evaluasi model menjadi lebih adil karena distribusi kelas target tidak berubah secara ekstrem akibat proses pembagian data.

Evaluasi model dilakukan menggunakan beberapa metrik, yaitu akurasi, F1 Score, Matthews Correlation Coefficient, Area Under Precision-Recall Curve, Area Under Receiver Operating Characteristic Curve, dan confusion matrix. Penggunaan beberapa metrik diperlukan karena akurasi saja belum cukup untuk menilai performa model pada data klasifikasi yang tidak sepenuhnya seimbang.

Tabel 4.2 Ruang Parameter dan Parameter Terbaik Random Forest

Hyperparameter	Variasi Parameter	Parameter Terbaik
n_estimators	[100, 200, 300, 500, 800]	500
max_depth	[10, 20, 30, 40, 60, 80, None]	30
min_samples_split	[2, 5, 10]	2
min_samples_leaf	[1, 2, 4]	1
max_features	['sqrt', 'log2']	log2
bootstrap	[True, False]	False

Sumber: Hasil pengolahan data penelitian.

Hasil tuning menunjukkan bahwa kombinasi parameter terbaik Random Forest adalah n_estimators=500, max_depth=30, min_samples_split=2, min_samples_leaf=1, max_features=log2, dan bootstrap=False. Parameter tersebut digunakan sebagai model Random Forest tuned.

4.5.2 Tuning XGBoost

Optimasi hyperparameter XGBoost dilakukan menggunakan GridSearchCV dengan 5-fold cross validation dan metrik akurasi. Grid Search digunakan karena ruang parameter XGBoost yang diuji lebih terbatas dibandingkan Random Forest.

Parameter yang diuji meliputi jumlah estimator, kedalaman maksimum pohon, learning rate, subsample, colsample_bytree, dan gamma. Parameter tersebut memengaruhi kompleksitas model, kecepatan pembelajaran, serta kemampuan model dalam mengurangi overfitting.

Tabel 4.3 Ruang Parameter dan Parameter Terbaik XGBoost

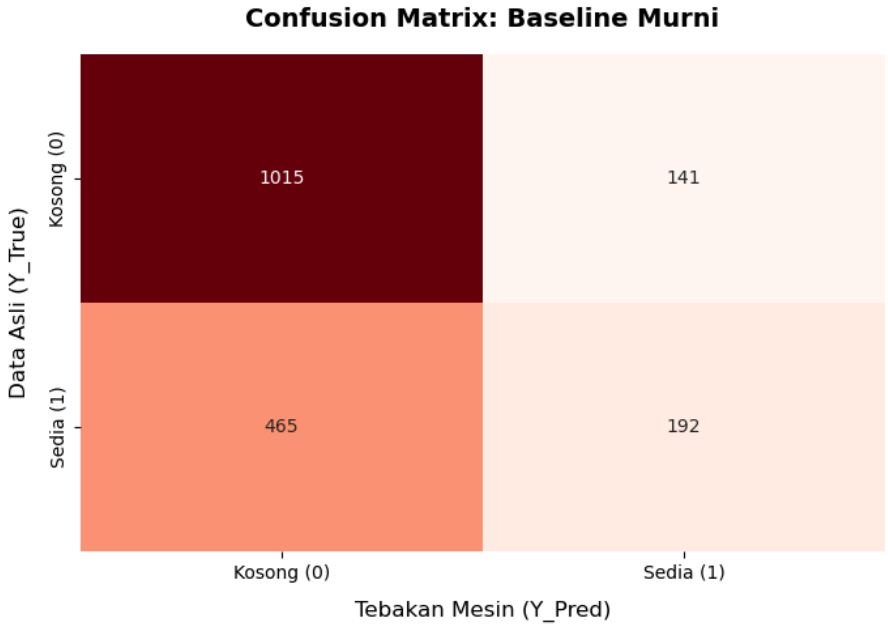
Hyperparameter	Variasi Parameter	Parameter Terbaik
n_estimators	[100, 300, 500]	300
max_depth	[3, 5, 7]	7
learning_rate	[0.01, 0.05, 0.1]	0.01
subsample	[0.8, 1.0]	1.0
colsample_bytree	[0.8, 1.0]	0.8
gamma	[0, 0.1, 0.2]	0.2

Sumber: Hasil pengolahan data penelitian.

Hasil tuning menunjukkan bahwa parameter terbaik XGBoost pada skema stratify=y adalah n_estimators=300, max_depth=7, learning_rate=0.01, subsample=1.0, colsample_bytree=0.8, dan gamma=0.2.

Berdasarkan Tabel 4.4, Rando

RF Baseline, Stratify y Class weigth



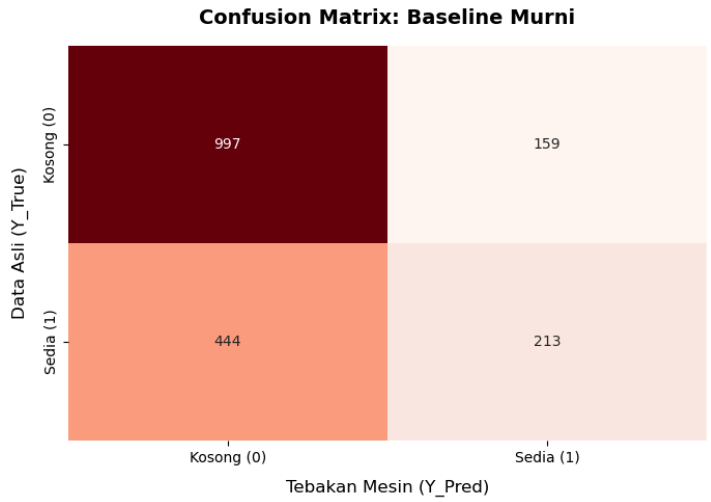
==========
==========
📊 HASIL EVALUASI BASELINE MURNI (CLASS WEIGHTED)
==========
==========
Rata-rata CV Score (5-Fold) : 0.6756
Akurasi Test : 0.6657
MCC Test : 0.2114
AUCPRC Test : 0.5387
AUROC Test : 0.6933
F1 Score Test : 0.3879

Nama_Kolom	Importance
B4R14	0.081167
B3R11E	0.078684
B9R13A	0.070291
B14R23	0.067231
B4R16A	0.046403

RF Baseline, Stratify y

📊 HASIL EVALUASI BASELINE MURNI (NO CLASS WEIGHT)
Rata-rata CV Score (5-Fold) : 0.6791
Akurasi Test : 0.6674
MCC Test : 0.2222
AUCPRC Test : 0.5443
AUROC Test : 0.6933
F1 Score Test : 0.4140

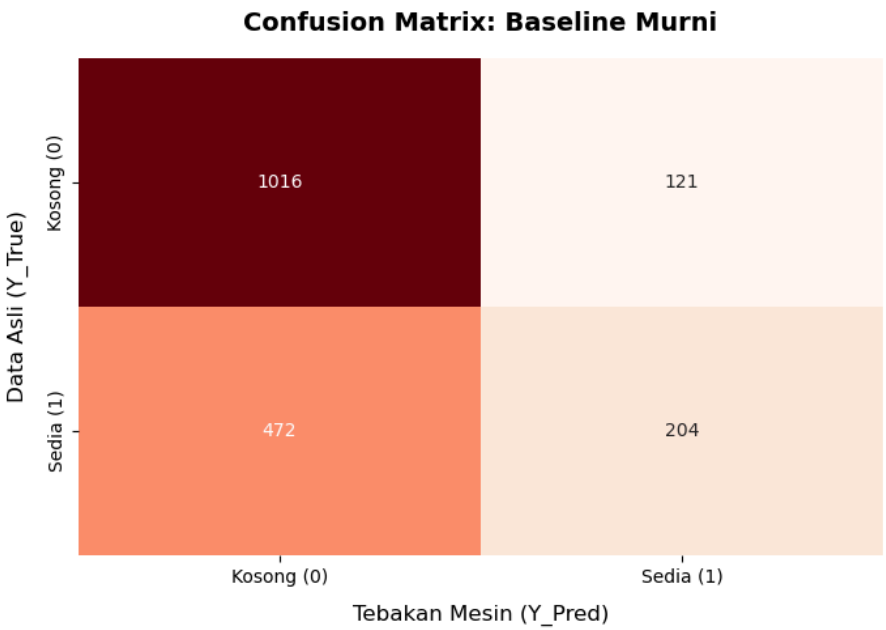
Nama_Kolom Importance



B4R14 0.081348
B3R11E 0.077816
B14R23 0.069231
B9R13A 0.068975

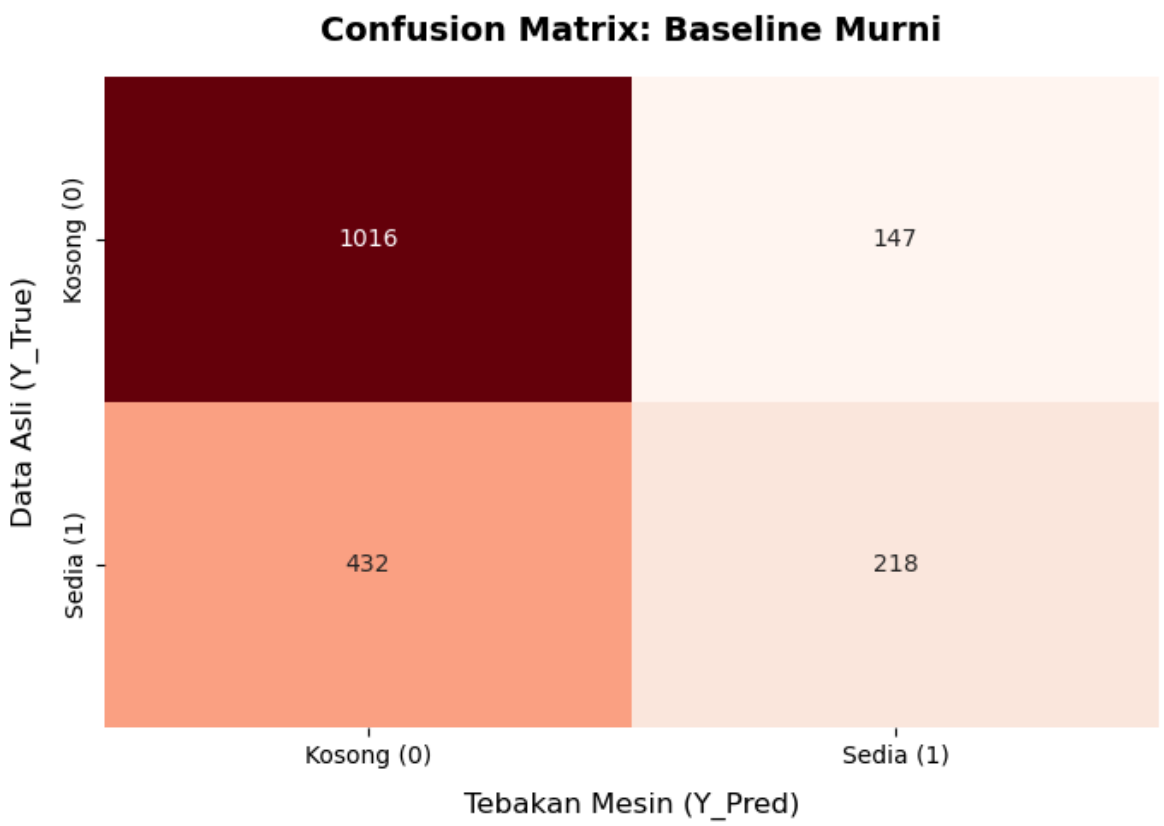
RF Baseline, Stratify B1R1

📊 HASIL EVALUASI BASELINE MURNI
Rata-rata CV Score (5-Fold) : 0.6751
Akurasi Test : 0.6729
MCC Test : 0.2463
AUCPRC Test : 0.5512
AUROC Test : 0.6819
F1 Score Test : 0.4076
🌟 5 FITUR PALING BERPENGARUH SEKARANG:
Nama_Kolom Importance
B4R14 0.082314
B3R11E 0.076880
B14R23 0.069531
B9R13A 0.068766
B4R16A 0.047474



RF RF Baseline, Stratify None

==========
==========
📊 HASIL EVALUASI BASELINE MURNI (CLASS WEIGHTED)
==========
==========
Rata-rata CV Score (5-Fold) : 0.6747
Akurasi Test : 0.6806
MCC Test : 0.2499
AUCPRC Test : 0.5623
AUROC Test : 0.6978
F1 Score Test : 0.4296
==========

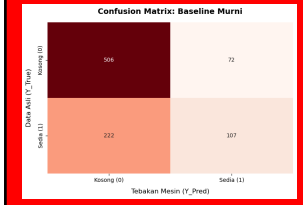


==========

🌟 5 FITUR PALING BERPENGARUH SEKARANG:

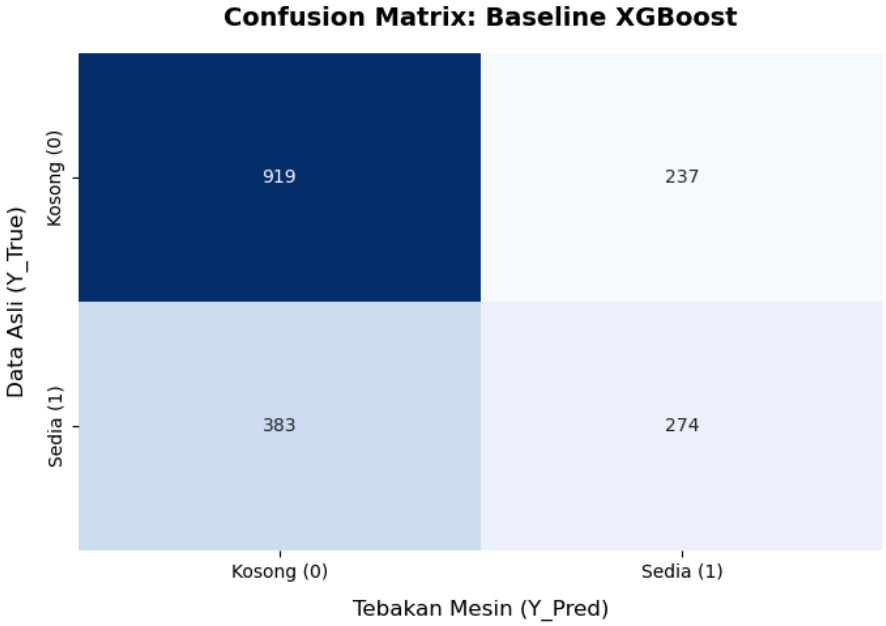
Nama_Kolom	Importance
B4R14	0.081013
B3R11E	0.077698
B14R23	0.069838

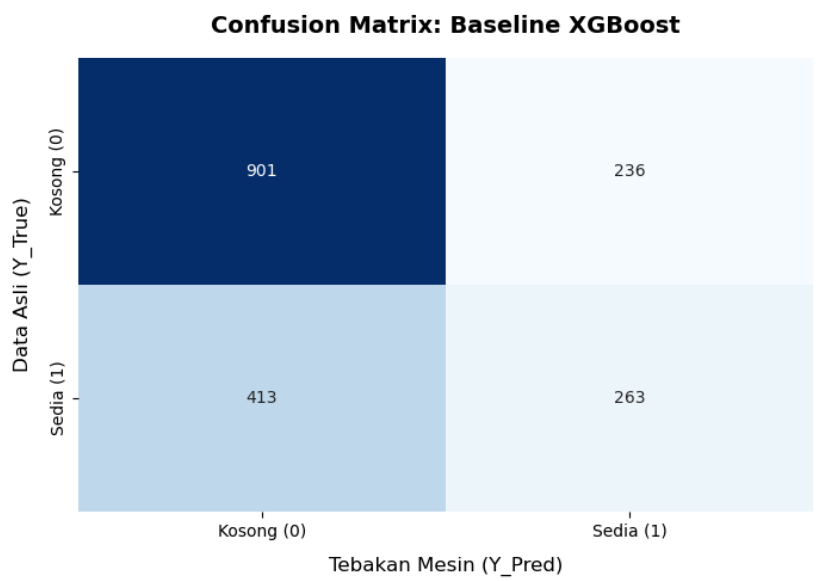
RF Baseline, 90:10 stratify y



📊 HASIL EVALUASI BASELINE MURNI (CLASS WEIGHTED)

Rata-rata CV Score (5-Fold) : 0.6775
Akurasi Test : 0.6759
MCC Test : 0.2424
AUCPRC Test : 0.5738
AUROC Test : 0.7125

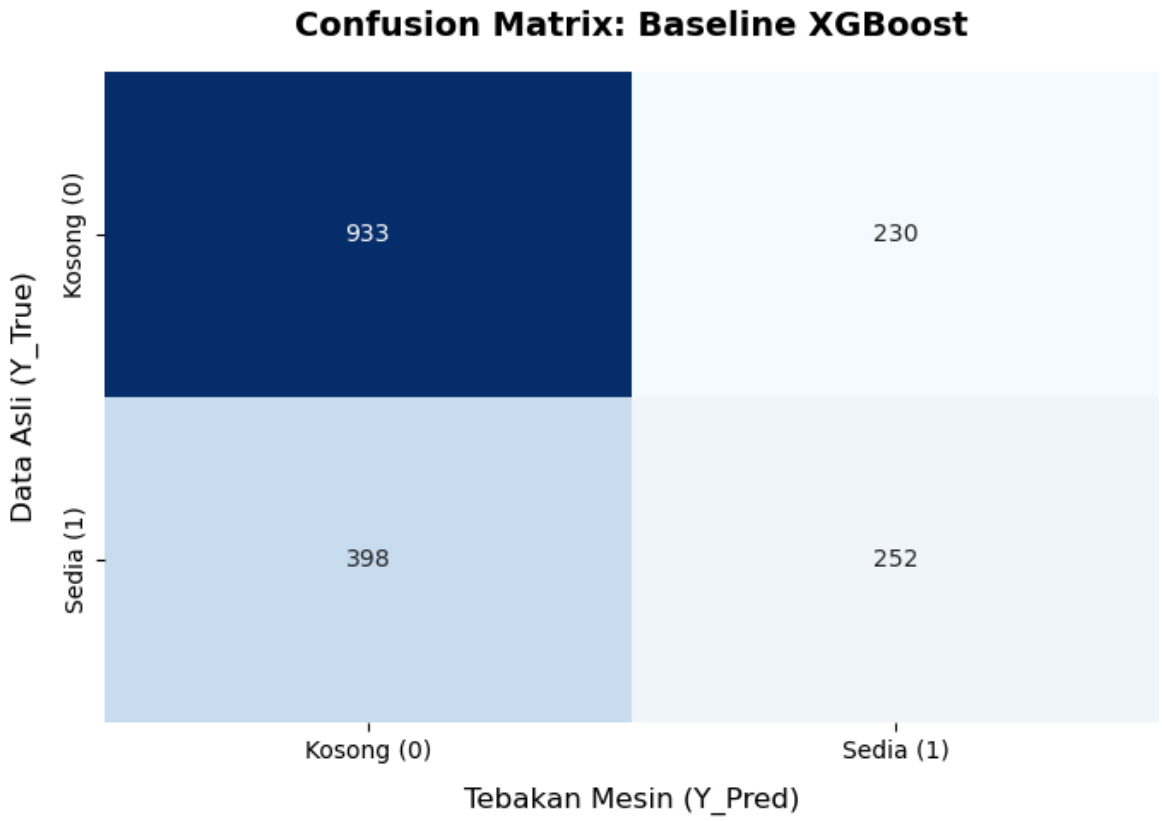
	F1 Score Test : 0.4213
Xgboost Baseline stratify y  	========== ========== 📊 HASIL EVALUASI BASELINE XGBOOST MURNI ========== ========== Rata-rata CV Score (5-Fold) : 0.6421 Akurasi Test : 0.6580253723110866 MCC Test : 0.2265388719491158 AUCPRC Test : 0.518675222486972 AUROC Test : 0.6690314052024248 F1 Score Test : 0.4691780821917808 ========== ========== 🌟 5 FITUR PALING BERPENGARUH (XGBOOST BASELINE): Nama_Kolom Importance X_Ketersediaan_Dinkes 0.037278 B14R7A 0.037127 B3R9B 0.035525 B9R13A 0.032192 X_Kepemilikan_Kendaraan 0.026500
Xgboost Baseline stratify B1R1	========== ========== 📊 HASIL EVALUASI BASELINE XGBOOST MURNI ========== ========== Rata-rata CV Score (5-Fold) : 0.6512 Akurasi Test : 0.6420297848869277 MCC Test :




```
0.19649753446813606
AUCPRC Test                 :
0.5161793380921378
AUROC Test                  :
0.6574149245653204
F1 Score Test              :
0.44765957446808513
==========================================
=======

🌟 5 FITUR PALING BERPENGARUH (XGBOOST
BASELINE):
           Nama_Kolom  Importance
X_Ketersediaan_Dinkes    0.038694
                B3R9B    0.034399
               B9R13A    0.031697
              B1282R3    0.029689
  X_Proporsi_Kapitasi    0.026372
```

Xgboost Baseline stratify none/random sampling

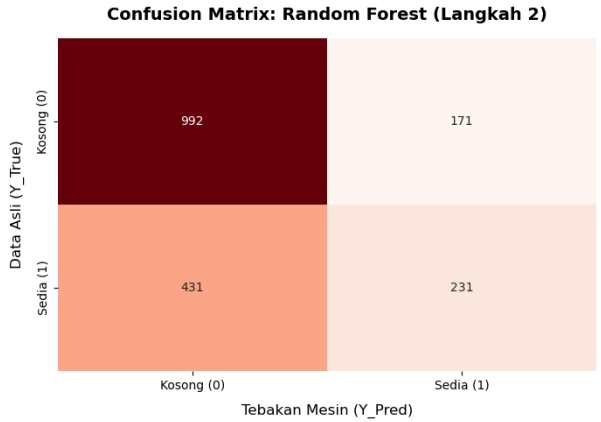
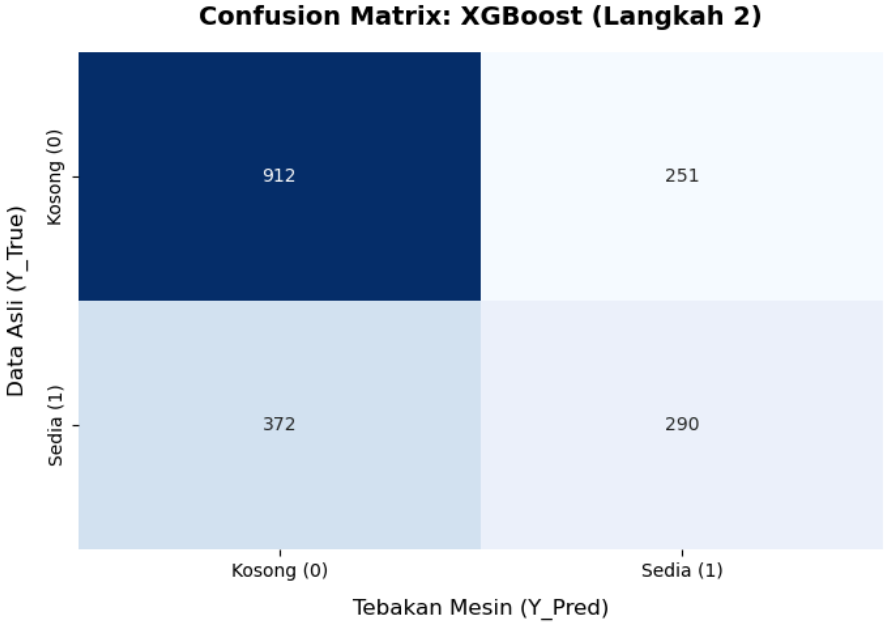
```
📊 HASIL EVALUASI BASELINE XGBOOST MURNI
==========================================
=======
Rata-rata CV Score (5-Fold) : 0.6432
Akurasi Test                :
0.6536127964699393
MCC Test                    :
0.2061687165095131
AUCPRC Test                 :
0.5150131883858395
AUROC Test                  :
0.6584020107149944
F1 Score Test              :
0.4452296819787986
==========================================
=======

🌟 5 FITUR PALING BERPENGARUH (XGBOOST
BASELINE):
           Nama_Kolom  Importance
```

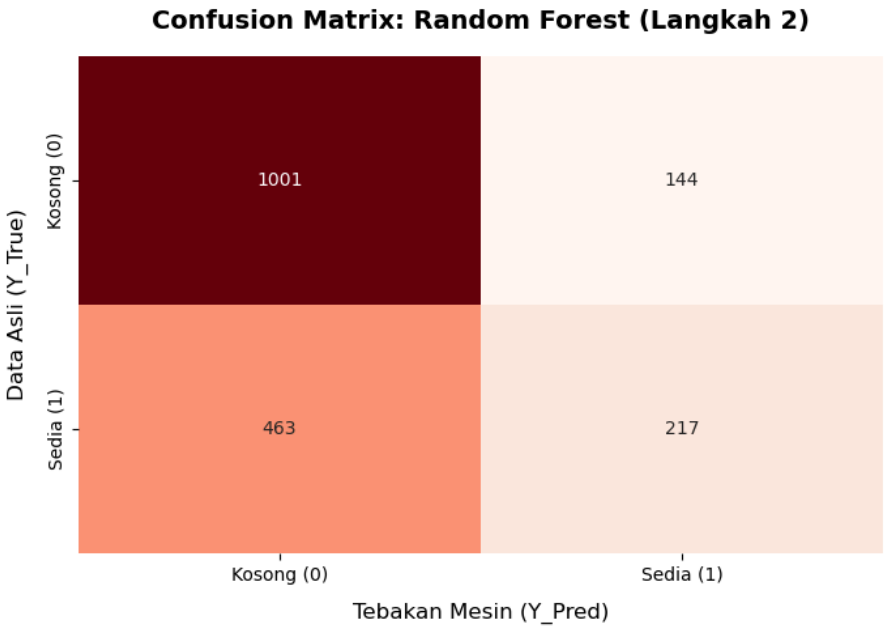


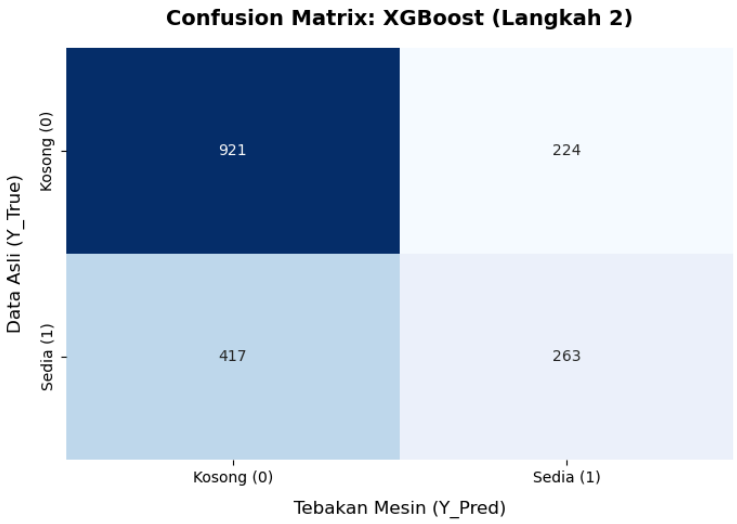

```
X_Ketersediaan_Dinkes    0.057723
               B9R13A    0.032006
              B1282R6    0.030388
                B3R9B    0.028762
  X_Proporsi_Kapitasi    0.026285
```

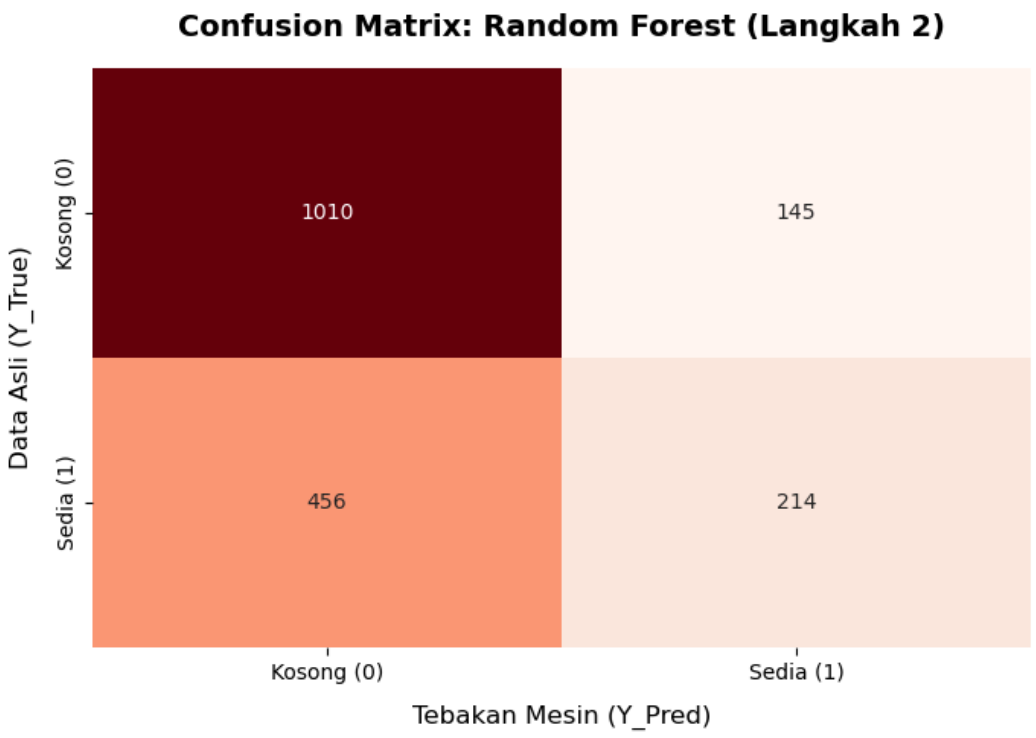
RF Baseline stratify y +Encoding + Scaling

🌲 RANDOM FOREST DEFAULT (WITH ENCODING):
Rata-rata CV Score (5-Fold) : 0.6780
- Akurasi Test : 0.6701
- F1-Score : 0.4342
- MCC Test : 0.2342
- AUPRC : 0.5359

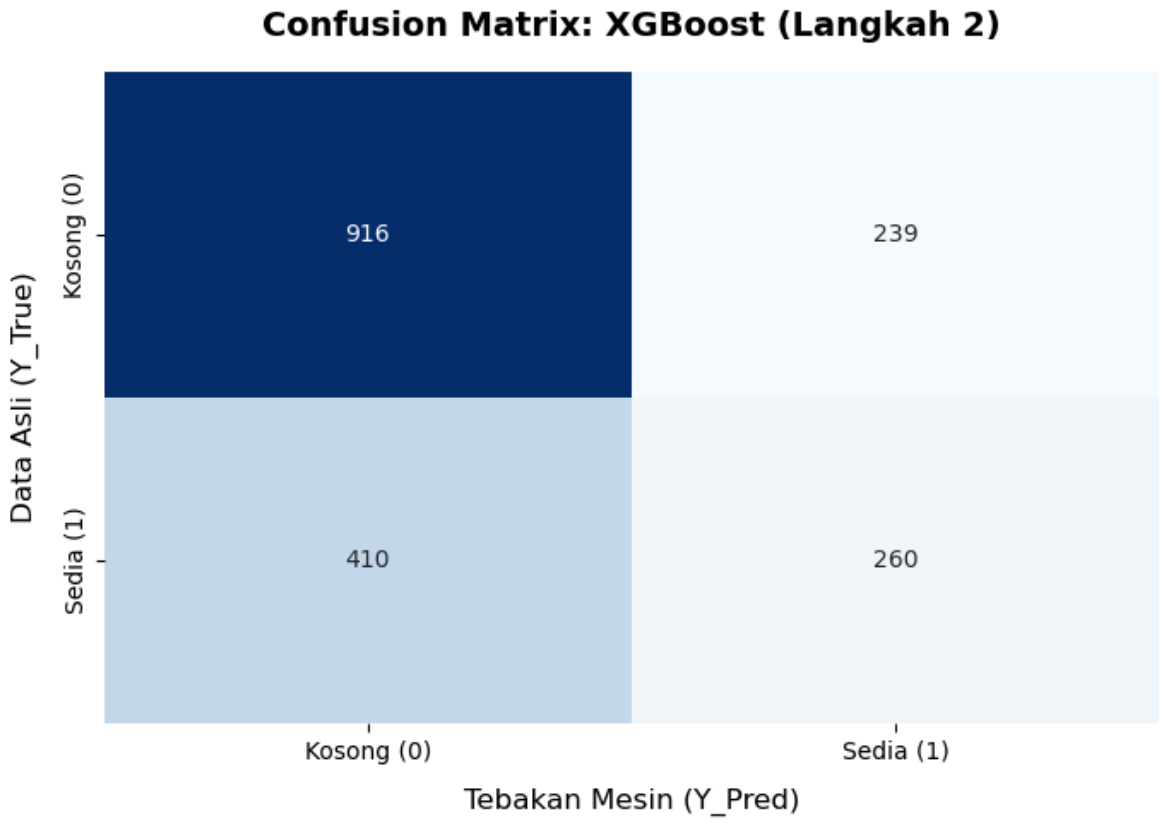
	- AUROC Test : 0.6902
Xgboost Baseline stratify y Encoding scaling 	Rata-rata CV Score (5-Fold) : 0.6526 - Akurasi Test : 0.6586 - F1-Score : 0.4821 - MCC Test : 0.2340 - AUROC Test : 0.6884 - AUPRC : 0.5365

RF Baseline stratify b1r1 +Encoding + Scaling  	Rata-rata CV Score (5-Fold) : 0.6776 - Akurasi Test : 0.6674 - F1-Score : 0.4169 - MCC Test : 0.2347 - AUPRC : 0.5732 - AUROC Test : 0.7000
Xgboost Baseline stratify b1r1 Encoding scaling	🚀 XGBOOST DEFAULT (WITH ENCODING): Rata-rata CV Score (5-Fold) : 0.6529 - Akurasi Test : 0.6488 - F1-Score : 0.4507 - MCC Test : 0.2089 - AUROC Test : 0.6755 - AUPRC : 0.5324

	
RF Baseline +Encoding + NO Scaling stratify none/random	🌲 RANDOM FOREST DEFAULT (WITH ENCODING): Rata-rata CV Score (5-Fold) : 0.6755 - Akurasi Test : 0.6707 - F1-Score : 0.4159 - MCC Test : 0.2351 - AUPRC : 0.5464 - AUROC Test : 0.6936

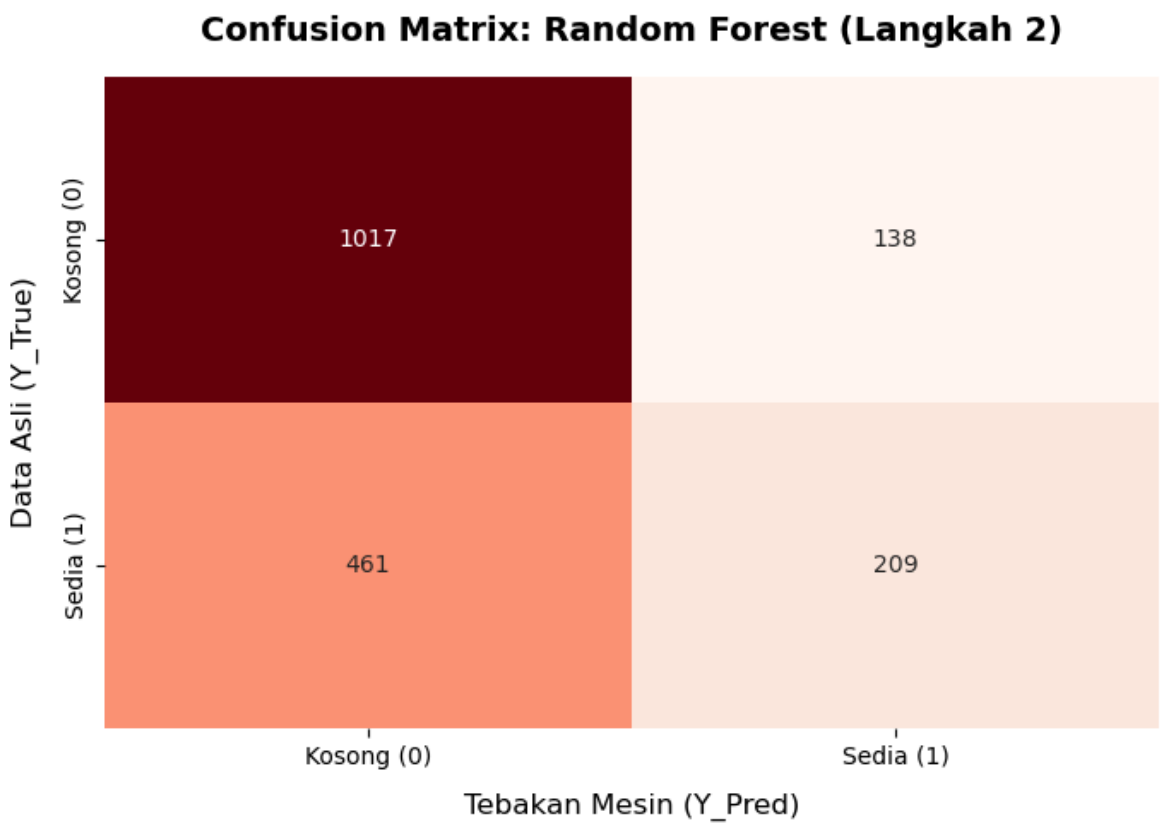

Confusion Matrix: Random Forest (Langkah 2)
Data Asli (Y_True)
Kosong (0)
Sedia (1)
1010
145
456
214
Kosong (0)
Sedia (1)
Tebakan Mesin (Y_Pred)

Xgboost +Encoding + NO Scaling stratify none/random

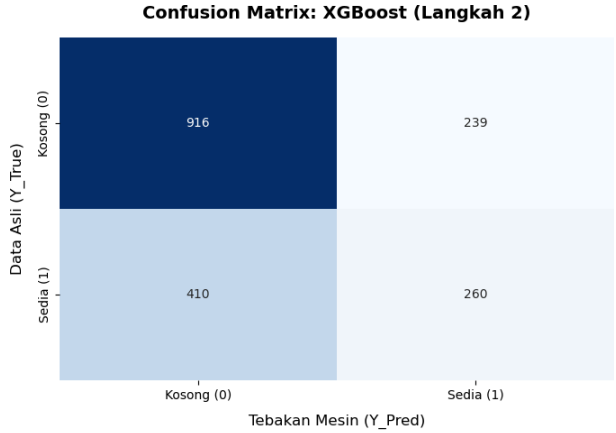


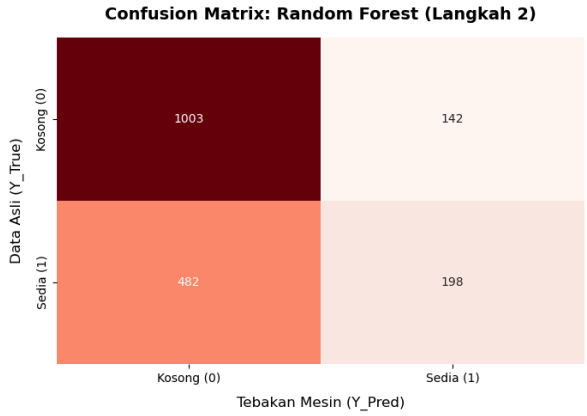
Rata-rata CV Score (5-Fold) : 0.6562
- Akurasi Test : 0.6444
- F1-Score : 0.4448
- MCC Test : 0.1959
- AUROC Test : 0.6784
- AUPRC : 0.5371

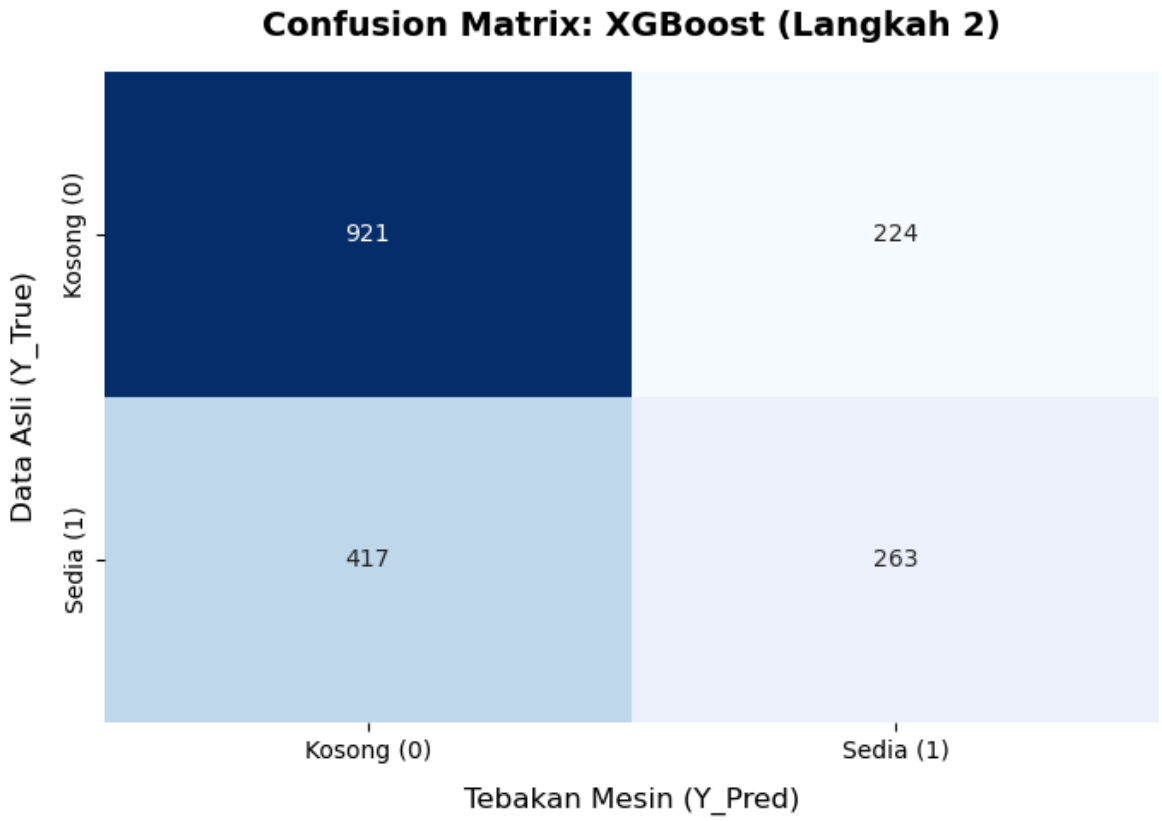
RF Baseline +Encoding + Scaling stratify none/random



🌲 RANDOM FOREST DEFAULT (WITH ENCODING):
Rata-rata CV Score (5-Fold) : 0.6781
- Akurasi Test : 0.6718
- F1-Score : 0.4110
- MCC Test : 0.2364
- AUPRC : 0.5561
- AUROC Test : 0.6933

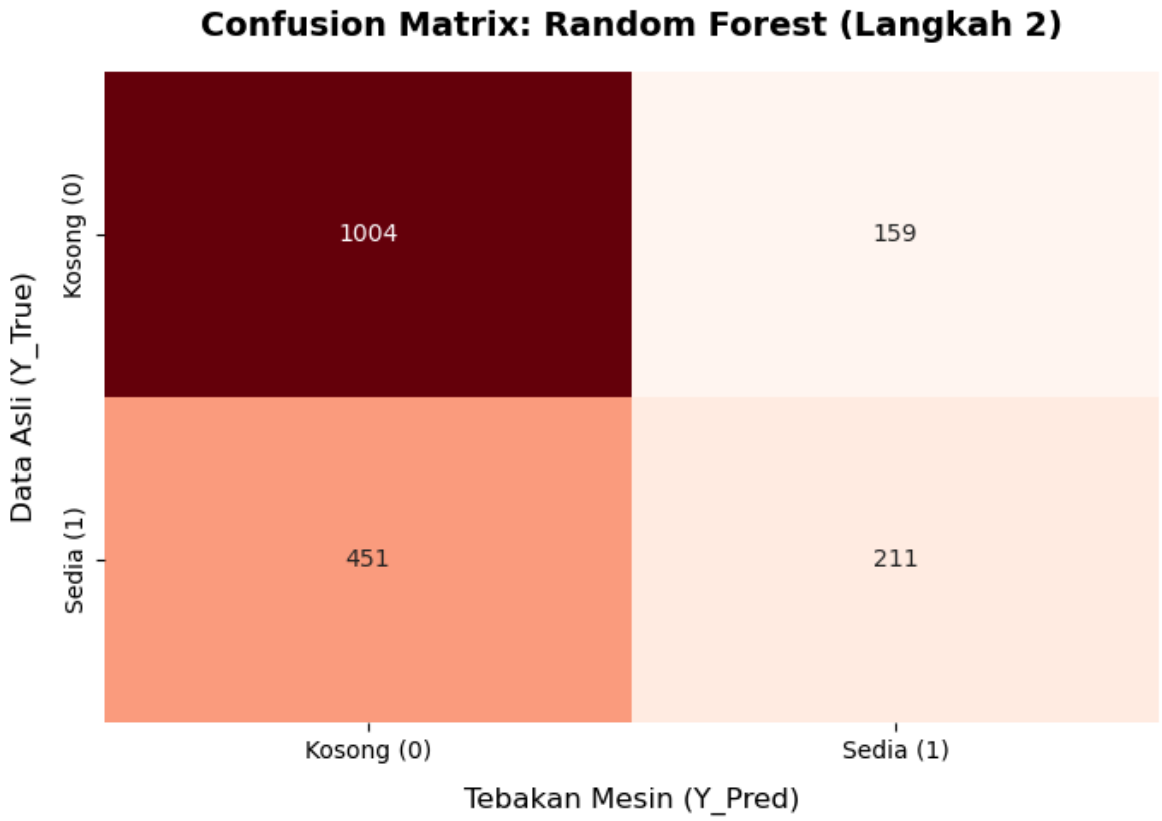
Xgboost +Encoding + NO Scaling stratify none/random  	Rata-rata CV Score (5-Fold) : 0.6562 - Akurasi Test : 0.6444 - F1-Score : 0.4448 - MCC Test : 0.1959 - AUROC Test : 0.6784 - AUPRC : 0.5371
RF Baseline stratify b1r1 +Encoding Tanpa Scaling Numerik	RANDOM FOREST DEFAULT (WITH ENCODING): Rata-rata CV Score (5-Fold) : 0.6770 - Akurasi Test : 0.6581 - F1-Score : 0.3882 - MCC Test : 0.2076 - AUPRC : 0.5614 - AUROC Test : 0.6982

	
Xgboost Baseline stratify b1r1 +Encoding Tanpa Scaling Numerik	🚀 XGBOOST DEFAULT (WITH ENCODING): Rata-rata CV Score (5-Fold) : 0.6529 - Akurasi Test : 0.6488 - F1-Score : 0.4507 - MCC Test : 0.2089 - AUROC Test : 0.6755 - AUPRC : 0.5324



RF Baseline stratify y +Encoding Tanpa Scaling Numerik

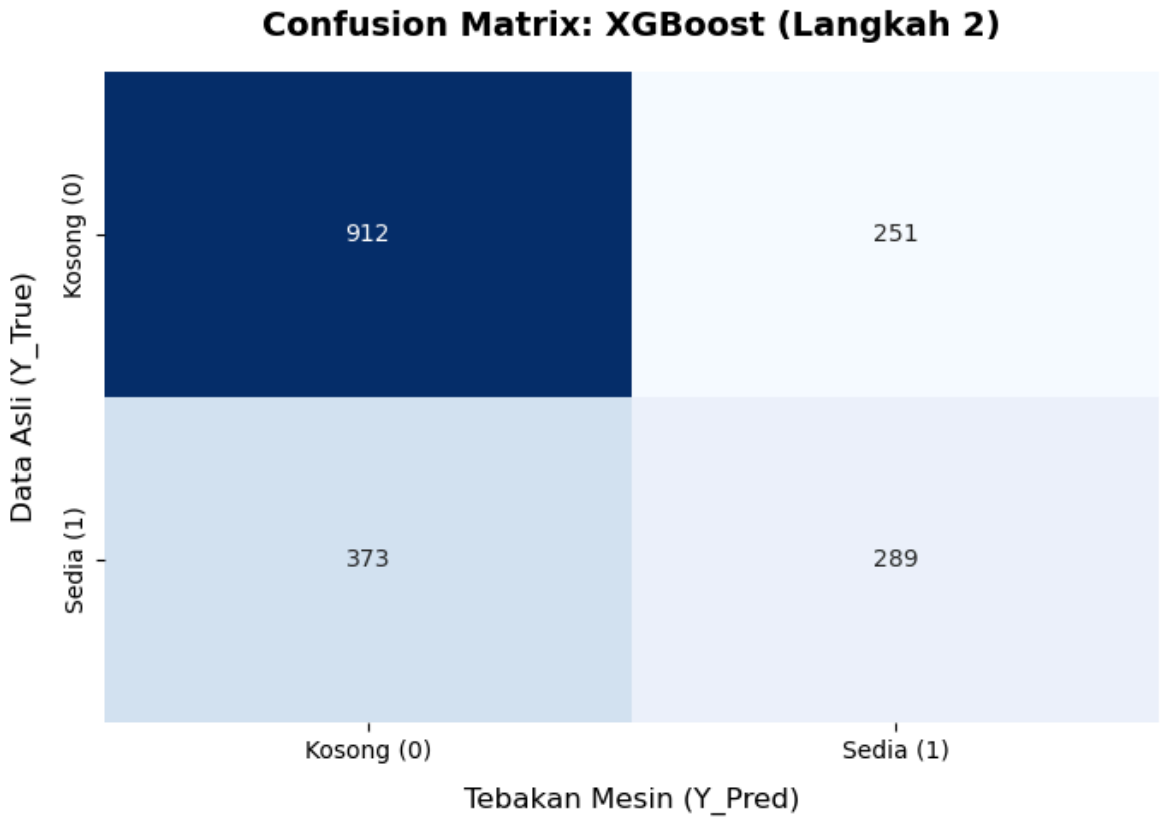
==========
==========
📊 HASIL LANGKAH 2: PREPROCESSING + MODEL DEFAULT
==========
==========



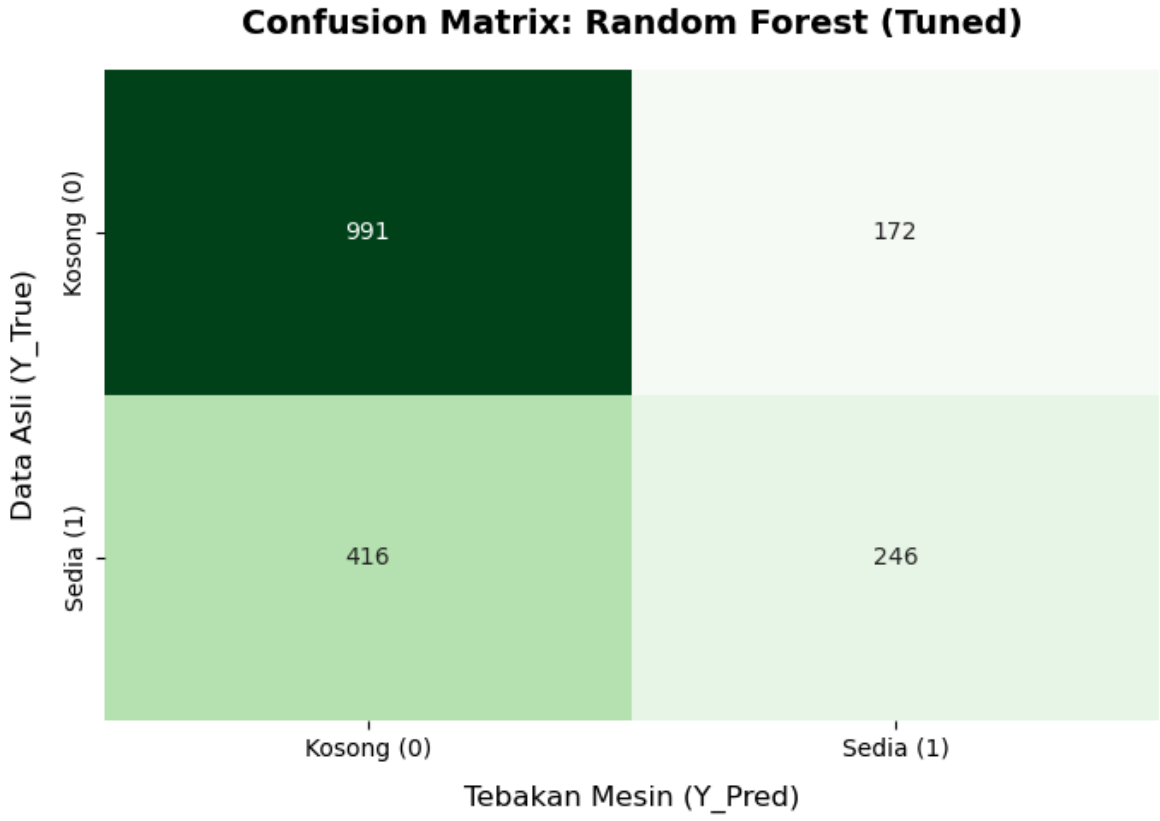
🌲 RANDOM FOREST DEFAULT (WITH ENCODING):
Rata-rata CV Score (5-Fold) : 0.6772
- Akurasi Test : 0.6658
- F1-Score : 0.4089
- MCC Test : 0.2177
- AUPRC : 0.5445
- AUROC Test : 0.6927

Xgboost Baseline stratify y +Encoding Tanpa Scaling Numerik

Rata-rata CV Score (5-Fold) : 0.6526
- Akurasi Test : 0.6581
- F1-Score : 0.4809
- MCC Test : 0.2325
- AUROC Test : 0.6883
- AUPRC : 0.5364


Confusion Matrix: XGBoost (Langkah 2)
Data Asli (Y_True)
Kosong (0)
Sedia (1)
912
251
373
289
Kosong (0)
Sedia (1)
Tebakan Mesin (Y_Pred)

randomized search RF + encoding scaling stratify



RANDOM SEARCH RANDOM FOREST SELESAI!

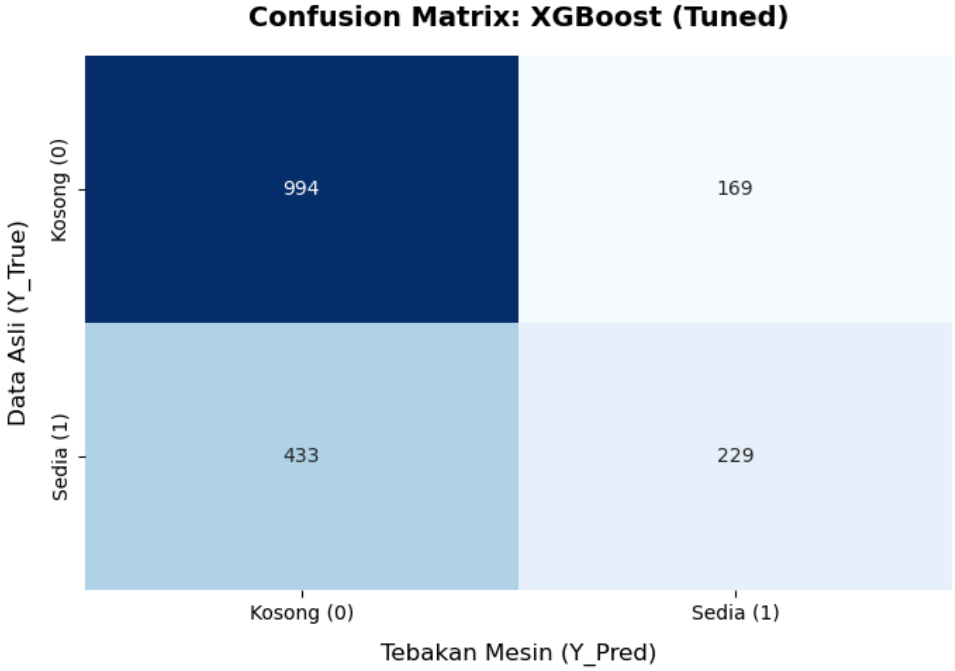
Skor Akurasi (CV): 0.6862
BEST CV SCORE: 0.6862163661568635
BEST PARAMETERS:
{'model__n_estimators': 500, 'model__min_samples_split': 2, 'model__min_samples_leaf': 1, 'model__max_features': 'log2', 'model__max_depth': 30, 'model__bootstrap': False}
==========
📊 HASIL EVALUASI RF TUNED (OTAK TERBAIK)
==========
CV Accuracy Final : 0.6813
Akurasi Test : 0.6778
MCC Test : 0.2560
AUCPRC Test : 0.5631
AUROC Test : 0.7076
F1 Score Test : 0.4556
==========

🌟 5 FITUR PALING BERPENGARUH (RF

TUNED):

	Nama_Kolom	Importance
	jalur_num__B4R14	0.064535
	jalur_num__B3R11E	0.063220
	jalur_num__B9R13A	0.058009
	jalur_num__B4R16A	0.041659
	jalur_num__B3R13A	0.037525

randomized Search Xgboost + encoding scaling stratify y



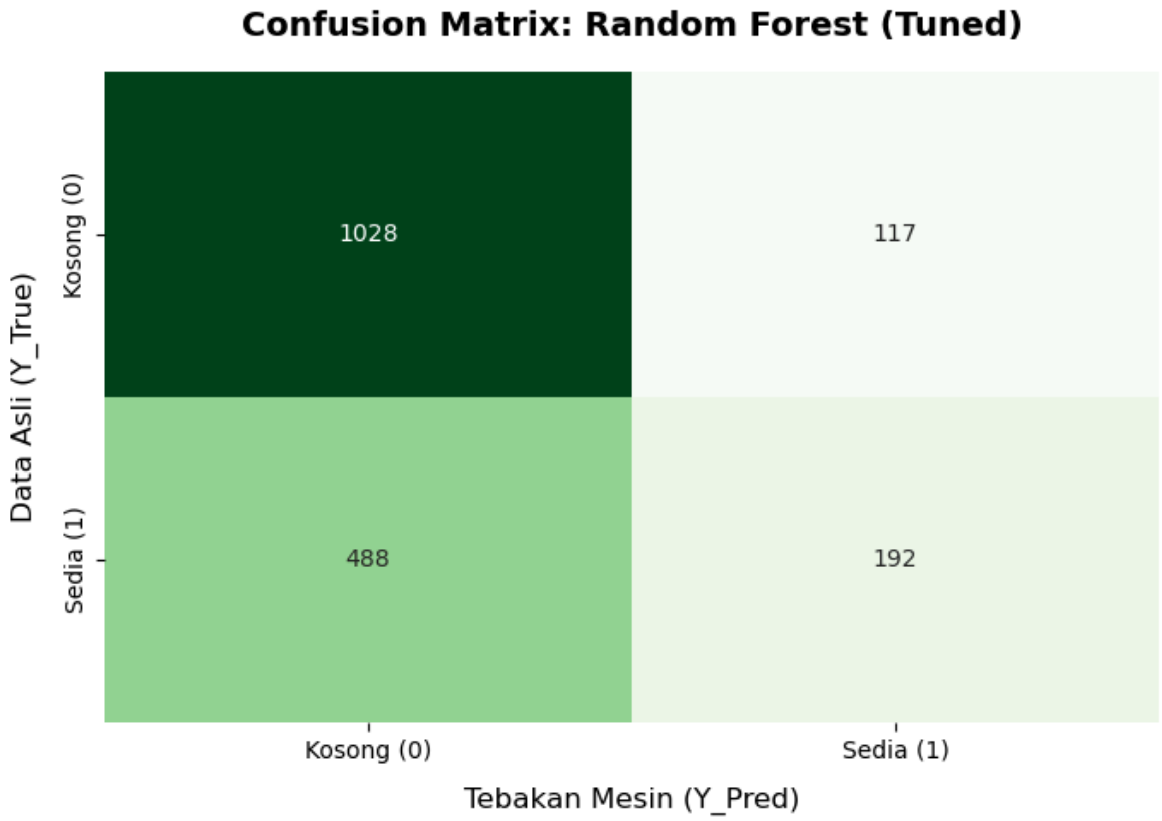
Parameter Terbaik :
{'model__colsample_bytree': 0.8, 'model__gamma': 0.2, 'model__learning_rate': 0.01, 'model__max_depth': 7, 'model__n_estimators': 300, 'model__subsample': 1.0}
🔥 Akurasi Terbaik : 0.6803

==========
📊 HASIL EVALUASI XGBOOST TUNED
==========

CV Accuracy Final : 0.6803
Akurasi Test : 0.6701
MCC Test : 0.2336
AUCPRC Test : 0.5618
AUROC Test : 0.7026
F1 Score Test : 0.4321
==========

🌟 5 FITUR PALING BERPENGARUH (XGBOOST TUNED):

	Nama_Kolom Importance remainder__B3R9B 0.043699 remainder__X_Ketersediaan_Dinkes 0.035287 jalur_num__B9R13A 0.033177 jalur_num__B4R16A 0.017646 remainder__B3R10 0.017362
randomized search RF + encoding scaling stratify B1R1	BEST CV SCORE: 0.683337457005646 BEST PARAMETERS: {'model__n_estimators': 500, 'model__min_samples_split': 5, 'model__min_samples_leaf': 1, 'model__max_features': 'log2', 'model__max_depth': 20, 'model__bootstrap': True} ========== 📊 HASIL EVALUASI RF TUNED (OTAK TERBAIK) ========== CV Accuracy Final : 0.6802 Akurasi Test : 0.6685 MCC Test : 0.2323 AUCPRC Test : 0.5815 AUROC Test : 0.7097 F1 Score Test : 0.3883 ========== 🌟 5 FITUR PALING BERPENGARUH (RF

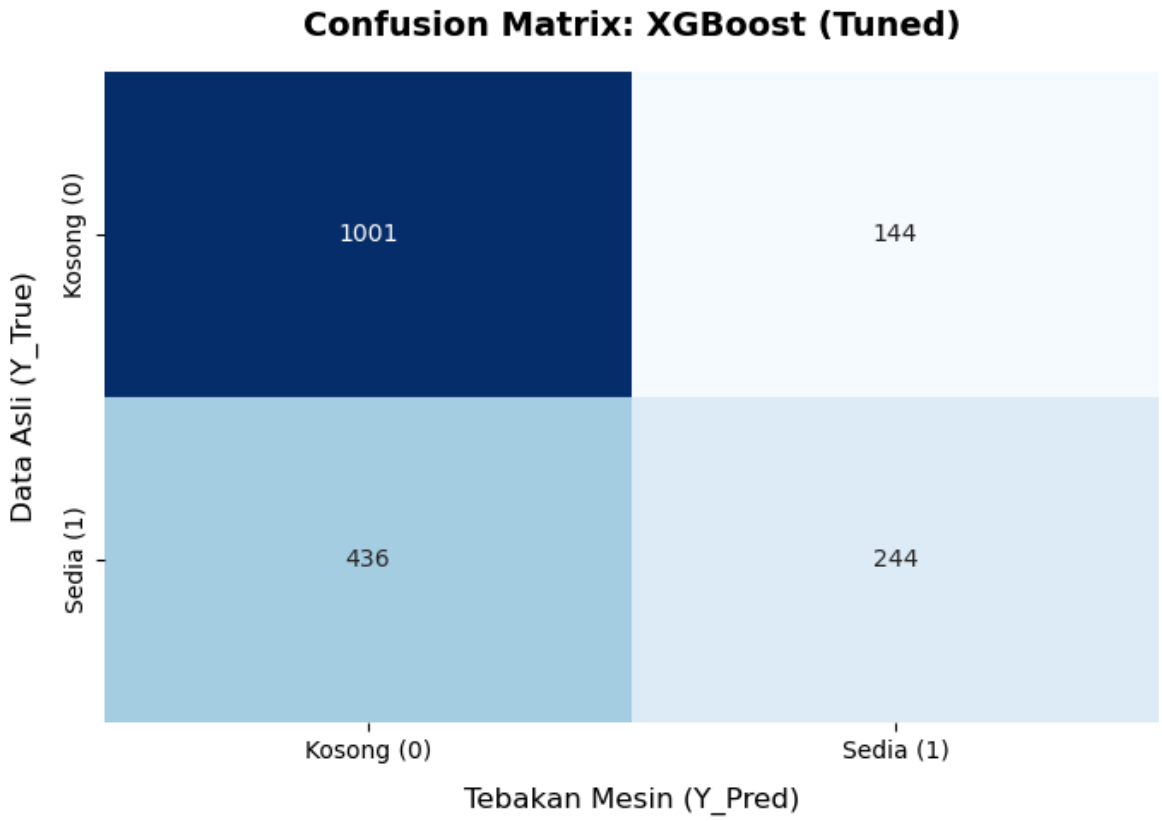


TUNED):

	Nama_Kolom	Importance
	jalur_num__B4R14	0.066717
	jalur_num__B3R11E	0.065579
	jalur_num__B9R13A	0.060301
	jalur_num__B4R16A	0.046937
	jalur_num__B3R13A	0.035408

randomized Search Xgboost + encoding scaling stratify B1R1

✅ XGBOOST SELESAI!
==========
🏆 Parameter Terbaik :
{'model__colsample_bytree': 1.0, 'model__gamma': 0, 'model__learning_rate': 0.01, 'model__max_depth': 5,

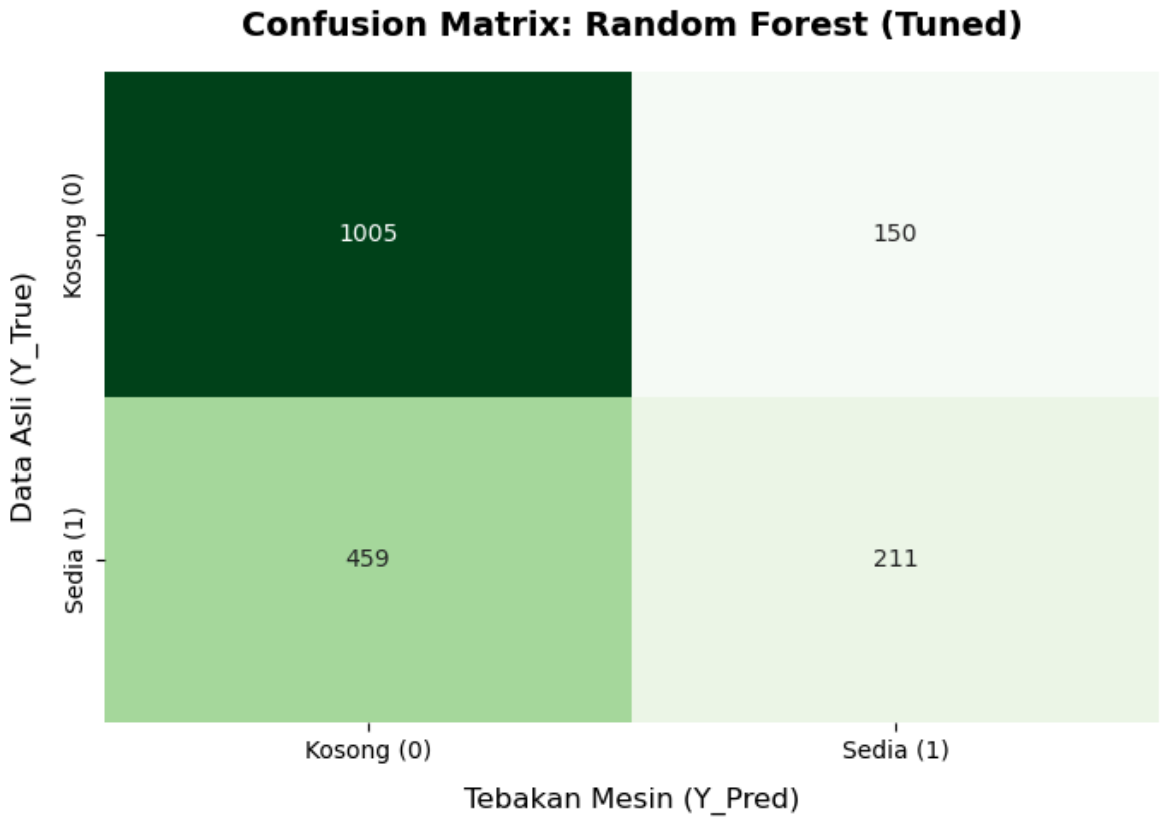


'model__n_estimators': 500,
'model__subsample': 0.8}
🔥 Akurasi Terbaik : 0.6806
==========
📊 HASIL EVALUASI XGBOOST TUNED
==========
CV Accuracy Final : 0.6806
Akurasi Test : 0.6822
MCC Test : 0.2754
AUCPRC Test : 0.5805
AUROC Test : 0.7118
F1 Score Test : 0.4569
==========

🌟 5 FITUR PALING BERPENGARUH (XGBOOST TUNED):
Nama_Kolom Importance
jalur_num__B9R13A 0.042314
remainder__B3R9B 0.033480
remainder__X_Ketersediaan_Dinkes 0.026693
jalur_nom__B14R7A_1 0.023098
remainder__X_Sedia_Obat_NonFornas 0.020119

randomized Search RF + ncoding scaling no stratify

✅ RANDOM SEARCH RANDOM FOREST SELESAI!
Skor Akurasi (CV): 0.6862
BEST CV SCORE: 0.6862163661568635



BEST PARAMETERS:
{'model__n_estimators': 500, 'model__min_samples_split': 2, 'model__min_samples_leaf': 1, 'model__max_features': 'log2', 'model__max_depth': 30, 'model__bootstrap': False}

==========

📊 HASIL EVALUASI RF TUNED (OTAK TERBAIK)

==========

CV Accuracy Final : 0.6810
Akurasi Test : 0.6663
MCC Test : 0.2239
AUCPRC Test : 0.5599
AUROC Test : 0.6993
F1 Score Test : 0.4093

==========

🌟 5 FITUR PALING BERPENGARUH (RF TUNED):

Nama_Kolom	Importance
jalur_num__B4R14	0.065400
jalur_num__B3R11E	0.065211
jalur_num__B9R13A	0.058203
jalur_num__B4R16A	0.040848
jalur_num__B3R13A	0.036526

Rrandomized Search Xgboost + encoding scaling no stratify

🏆 Parameter Terbaik :

	{'model__colsample_bytree': 0.8, 'model__gamma': 0.2, 'model__learning_rate': 0.05, 'model__max_depth': 7, 'model__n_estimators': 100, 'model__subsample': 1.0} 🔥 Akurasi Terbaik : 0.6827 ========== 📊 HASIL EVALUASI XGBOOST TUNED ========== CV Accuracy Final : 0.6827 Akurasi Test : 0.6718 MCC Test : 0.2426 AUCPRC Test : 0.5645 AUROC Test : 0.6953 F1 Score Test : 0.4365 ========== 🌟 5 FITUR PALING BERPENGARUH (XGBOOST TUNED): Nama_Kolom Importance jalur_num__B9R13A 0.033465 remainder__B3R9B 0.032086 remainder__X_Ketersediaan_Dinkes 0.031905 remainder__X_Sedia_Obat_NonFornas 0.019712 jalur_num__B4R16A 0.017715